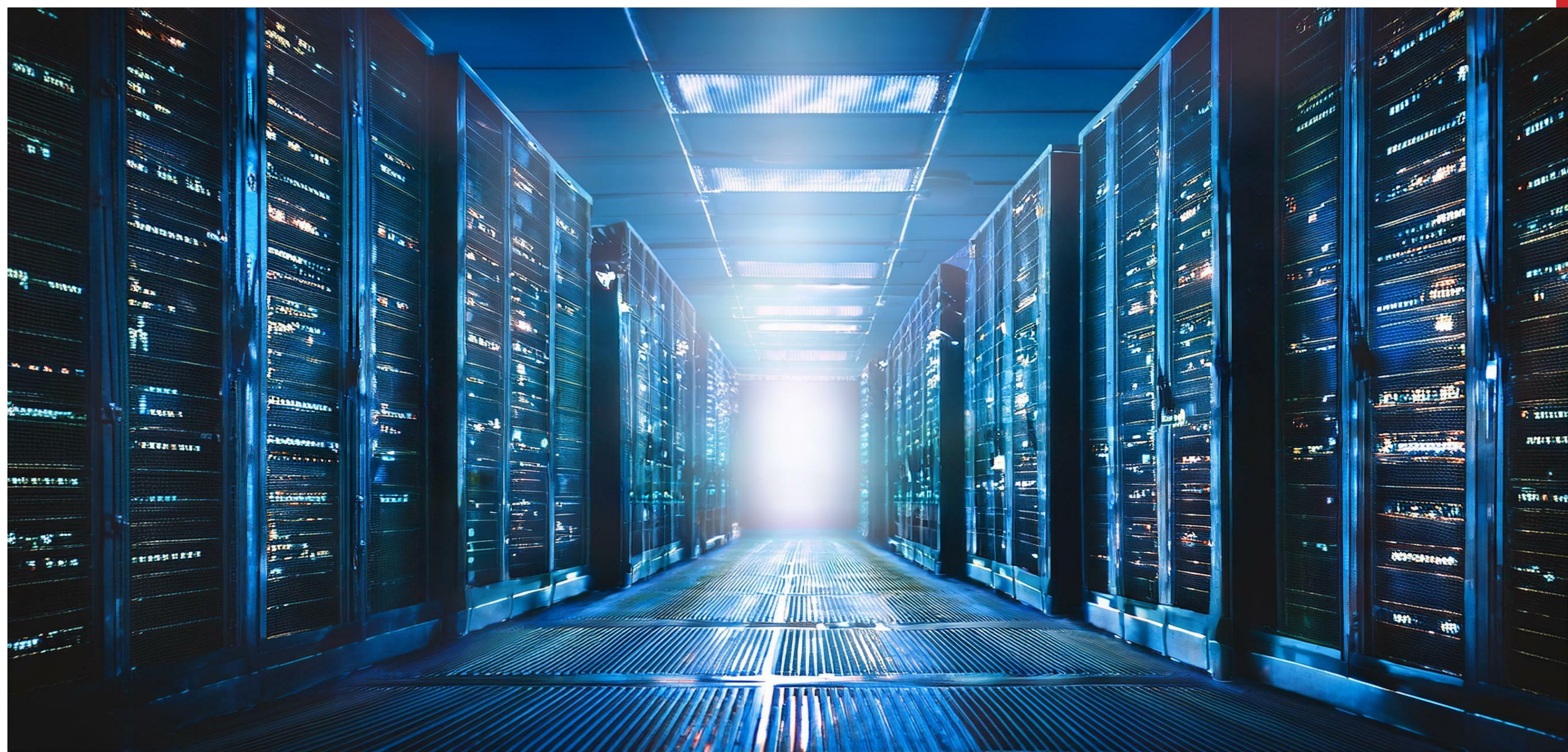


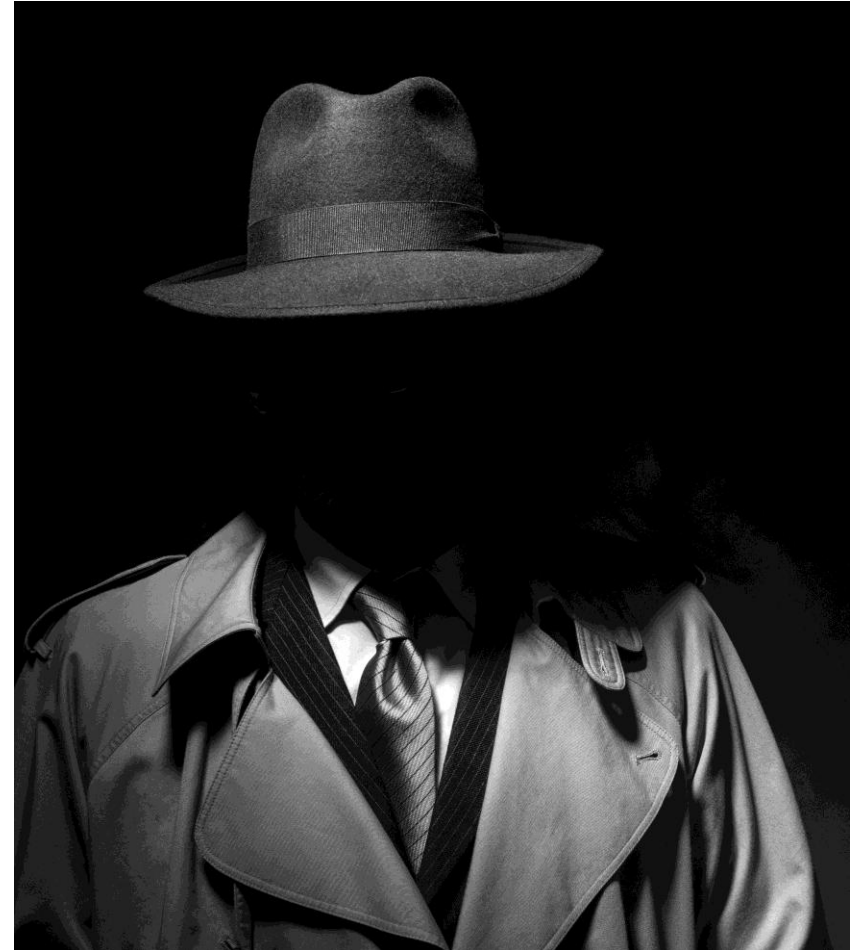
Cybersecurity in AI



FROM DETECTION TO INVESTIGATION — DAY 2 OVERVIEW

Day 1 built the foundation: a running EC2 instance, structured log evidence, AI-assisted scripting, and a cloud security posture baseline.

Day 2 connects those pieces into a live detection and investigation pipeline using AWS-native AI tooling.



FROM DETECTION TO INVESTIGATION

— DAY 2 OVERVIEW

What you will build today:

- Lab 2.2 — Query security logs with natural language and run AI-assisted investigations using CloudWatch AI Operations
- Lab 2.3 — Build a real-time SOC dashboard with metric filters, alarms, and automated investigation triggers
- Lab 2.4 — Choose a real attack technique, simulate it, build the full detection pipeline independently, and write a structured incident report



WHAT IS A SECURITY INFORMATION AND EVENT MANAGEMENT SYSTEM?

A Security Information and Event Management (SIEM) system collects, normalizes, and correlates log data from across an environment to surface security-relevant events in one place.

SIEMs are the central tool in most SOC workflows. Analysts spend the majority of their time working within or alongside one.

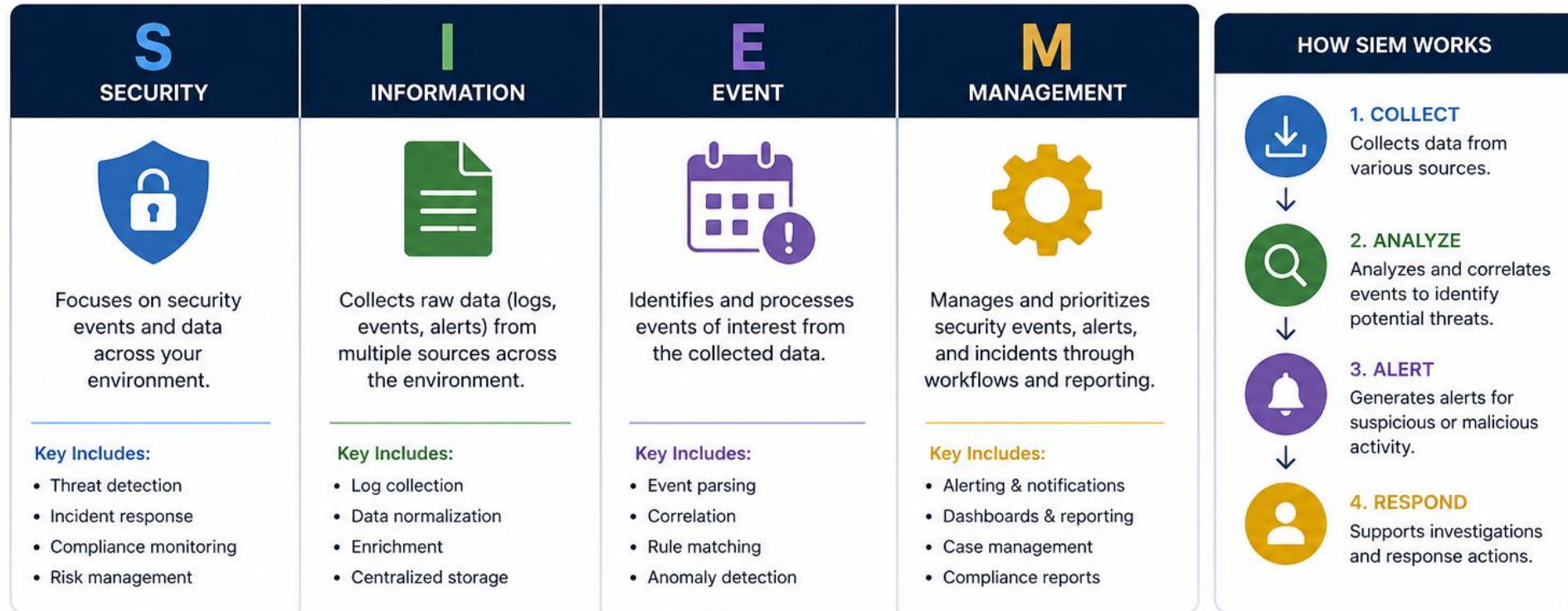
Core SIEM functions:

- Log aggregation — collect events from endpoints, network devices, cloud services, and applications
- Normalization — convert heterogeneous log formats into a consistent schema for querying
- Correlation rules — detect patterns across multiple event sources that no single source would surface alone
- Alerting — notify analysts when correlation rules or thresholds trigger
- Search and investigation — allow analysts to query historical data during incident response

SIEM

Security Information and Event Management

SIEM solutions collect, analyze, and correlate security data in real time to detect threats, support investigations, and ensure compliance.



DATA SOURCES

Servers, endpoints, firewalls, cloud, applications, IAM, network devices, and more.



SIEM PLATFORM

Collects, stores, and processes data in real time.



ANALYTICS & CORRELATION

Applies rules, ML, and threat intelligence to detect and prioritize threats.



ACTION & RESPONSE

Alerts, dashboards, and workflows enable fast response and remediation.



Better Visibility. Faster Detection. Smarter Response.

CLOUD-NATIVE MONITORING VS. TRADITIONAL SIEMS

Traditional SIEMs were designed for on-premises environments where logs had to be shipped over the network from physical devices.

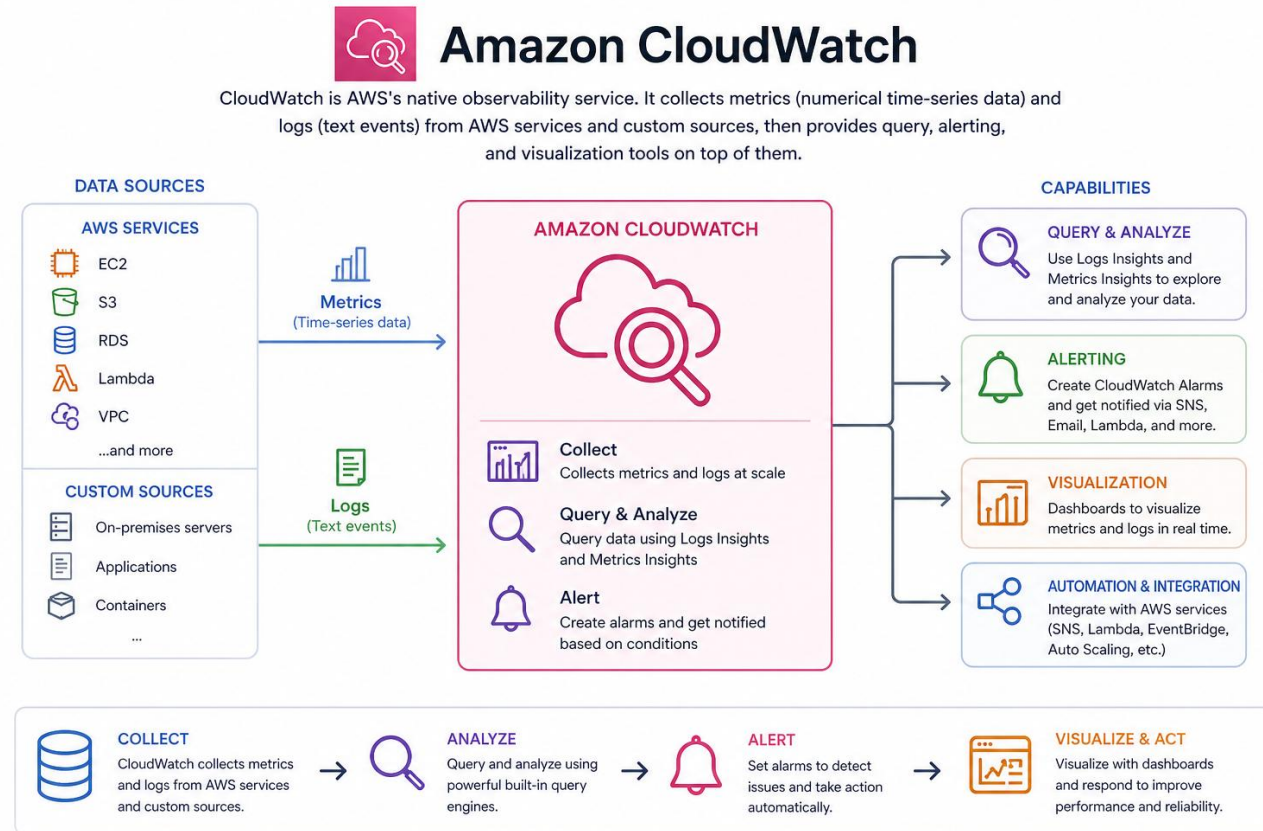
Cloud-native monitoring takes a different approach, logs are generated, stored, and analyzed within the same cloud infrastructure.

Dimension	Traditional SIEM	Cloud-Native (CloudWatch)
Log collection	Agent-based, network push	Native service integration
Scaling	Manual capacity planning	Automatic
Query language	SPL, KQL, vendor-specific	CloudWatch Logs Insights
Cost model	License + infrastructure	Pay per ingestion/query
AI integration	Add-on or third-party	Built into the service

AMAZON CLOUDWATCH — ARCHITECTURE OVERVIEW

CloudWatch is AWS's native observability service.

It collects metrics (numerical time-series data) and logs (text events) from AWS services and custom sources, then provides query, alerting, and visualization tools on top of them.



AMAZON CLOUDWATCH — ARCHITECTURE OVERVIEW

Key components:

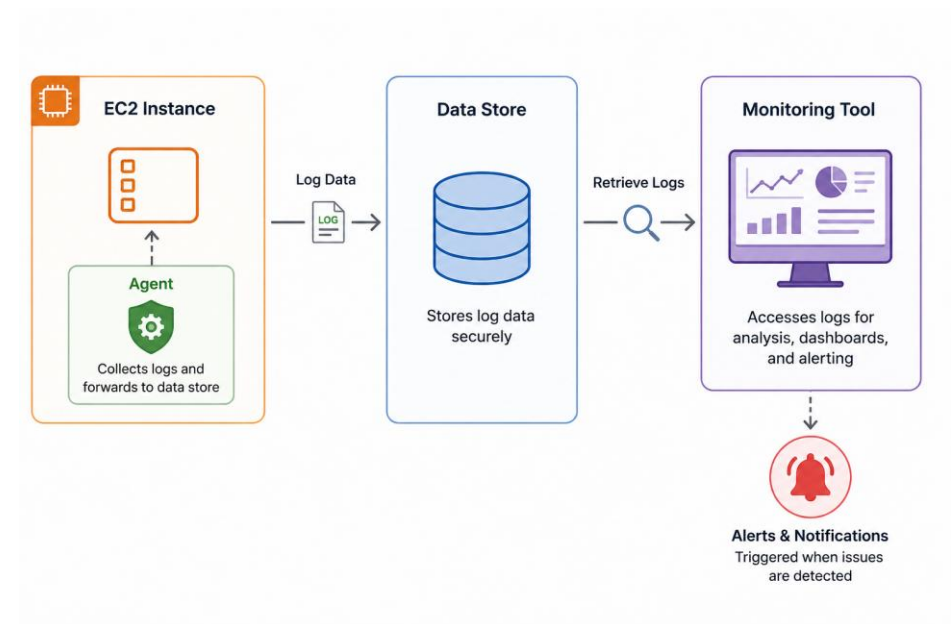
- Log groups — containers for related log streams (e.g., /soc-lab/secure for endpoint auth events)
- Log streams — individual sequences of log events from a single source (e.g., one EC2 instance)
- Metrics — numerical measurements over time (e.g., CPUUtilization, or custom SudoFailedAttempts)
- Alarms — monitors a metric against a threshold; triggers actions when breached
- Dashboards — configurable visualization panels combining metrics and log queries



THE CLOUDWATCH AGENT AND LOG INGESTION

The CloudWatch agent is a software process that runs on an EC2 instance and continuously ships log data to CloudWatch.

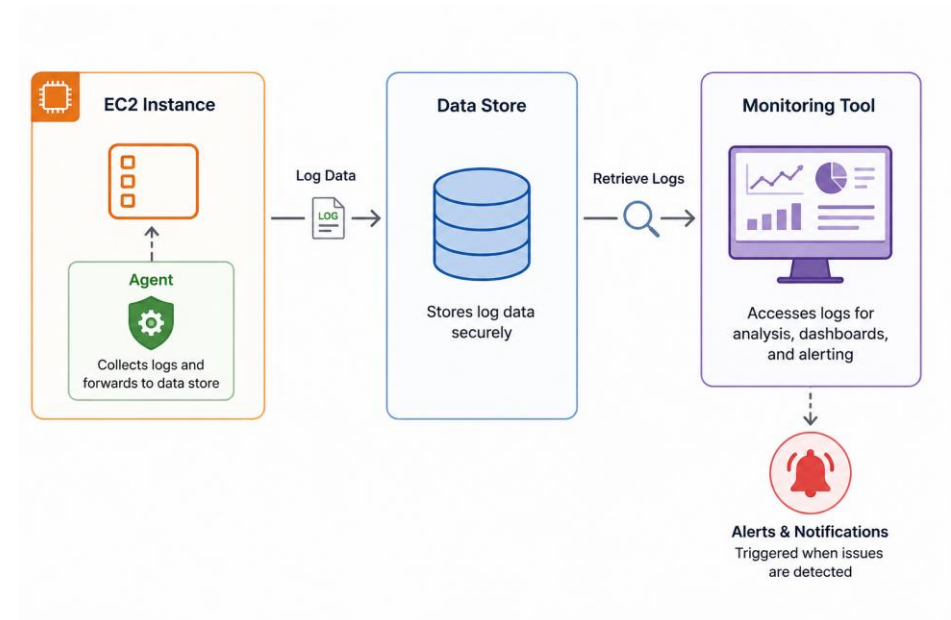
Without it, CloudWatch has no visibility into the host, only into AWS-managed services.



THE CLOUDWATCH AGENT AND LOG INGESTION

How log ingestion works:

- The agent runs on the EC2 instance as a system service
- A configuration file tells it which log files to watch and which log group to send them to
- The agent tails configured files and forwards new entries to CloudWatch in near real time
- Entries land in a log stream named after the instance ID
- All streams within one log group can be searched together via Logs Insights



CLOUDWATCH LOGS INSIGHTS — QUERY LANGUAGE

Logs Insights is the query engine for CloudWatch log data. It uses a structured query language purpose-built for log analysis. It is simpler than SQL, but powerful enough for most SOC investigation tasks.

Common query patterns:

fields — select which columns to display

filter — apply conditions (supports regex with like /pattern/)

stats — aggregate; count(), sum(), avg()

sort and limit — control output order and volume

```
# Filter for a keyword
fields @timestamp, @message
| filter @message like /Failed password/
| sort @timestamp desc
| limit 50

# Count events by source over time
fields @timestamp, @message
| filter @message like /exe="\usr\sbin\sshd"/
| stats count() by bin(5m)
```


AI-POWERED QUERY GENERATION IN CLOUDWATCH

CloudWatch Log Analytics includes an AI interface that converts plain English questions into Logs Insights query syntax. This lowers the barrier to log analysis for analysts who don't know the query language.

This does not replace query knowledge, it accelerates writing the first draft. Analysts still need to validate that the generated query matches the actual log structure in their environment.

The screenshot displays the CloudWatch Logs Insights console. At the top, the 'Query definition' section includes an 'Info' box about Log Analytics, time range filters (30m, 3h, 1h), a 'Compare (Off)' button, a 'UTC timezone' dropdown, and a 'Start tailing' button. Below this, the 'Query scope' section shows a 'Log group name' dropdown, a search bar with the text 'Select up to 50 log groups', and a 'Browse' link with sub-links for 'Log Groups', 'Facets', and 'Lookup tables'. A specific log group is selected: '/soc-lab/secure' with monitoring account '421377222628'. A 'Clear all' button is also present. The main area shows a query editor with the following syntax:

```
1 1 fields @timestamp, @message
2+ filter @message like /sudo/
2 3 sort @timestamp desc
3 - limit 10000
4+ limit 25
```

 Below the editor is an 'Accept changes' button with refresh and undo icons. A navigation bar contains 'Logs Insights QL', 'Query generator' (active), 'Fields', 'Saved and sample queries', and 'Query commands'. The 'Query generator' section has a 'Prompt' tab and an 'Info' tab. The prompt text is 'All logs containing sudo'. Below the prompt are 'Generate new query' and 'Refine existing query' buttons, along with a settings gear icon. A note states: 'The selected log groups are used as context when generating queries.' At the bottom, there are buttons for 'Run query', 'Cancel', 'Save', 'Schedule query', and 'History'.

AI-POWERED QUERY GENERATION IN CLOUDWATCH

1. You type a natural language prompt: `"Find failed SSH attempts in the last hour"`
2. The AI generates a complete Logs Insights query targeting the fields it infers are relevant
3. You review the generated query before running it — the AI output is editable
4. Results appear in the same interface as a manually written query

The screenshot displays the AWS CloudWatch Logs Insights 'Query definition' interface. At the top, there's an 'Info' banner for 'Log Analytics'. Below it, filters for '30m', '3h', '1h', and a 'Compare (Off)' button are visible. The 'Query scope' section shows 'Log groups' selected, with a dropdown menu displaying '/soc-lab/secure' and 'Monitoring account 421377222628'. A 'Clear all' button is next to it. The generated query is shown in a code editor with syntax highlighting: `1 fields @timestamp, @message`, `2+ filter @message like /sudo/`, `2 sort @timestamp desc`, `3 limit 10000`, and `4+ limit 25`. Below the query, there are buttons for 'Accept changes', 'Logs Insights QL', 'Query generator', 'Fields', 'Saved and sample queries', and 'Query commands'. A 'Prompt' section shows the input 'All logs containing sudo' with a character count of 24/1000. Below the prompt are buttons for 'Generate new query' and 'Refine existing query'. At the bottom, there are buttons for 'Run query', 'Cancel', 'Save', 'Schedule query', and 'History'.

CLOUDWATCH AI OPERATIONS — WHAT IT IS

AI Operations is a new CloudWatch feature that runs structured investigations over your log and metric data using AI.

Unlike a simple query, an investigation forms multiple hypotheses, evaluates evidence across data sources, and presents conclusions for analyst review.



CLOUDWATCH AI OPERATIONS — WHAT IT IS

[AI Operations: Investigations](#) > Failed SSH attempts

 Hypotheses

2

Internal SSH Security Scanning on soc-lab-ec2-monitoring: Automated internal security scanning process initiated SSH connection attempts to localhost on the soc-lab-ec2-monitoring service, generating systematic authentication failures for unknown accounts while maintaining proper cryptographic operations and audit logging. This represents legitimate security testing activity rather than malicious compromise, as evidenced by functional SSH security controls and correlation with decreased external attack activity.

> ⋮

1

Potential SSH Security Compromise in soc-lab-ec2-monitoring: Suspicious SSH authentication activity from localhost on the soc-lab-ec2-monitoring service indicates either a security compromise or severe misconfiguration, with multiple failed login attempts for unknown accounts suggesting unauthorized internal access attempts. The pattern of PAM authentication failures and cryptographic session attempts from 127.0.0.1 at 01:07:22 UTC requires immediate security assessment to determine if this represents system compromise or lateral movement attempts.

> ⋮

AI Operations is not a black box:

- Every hypothesis includes the evidence the AI cited
- The analyst reviews and accepts, rejects, or escalates each finding
- Investigations can be triggered manually or automatically by a CloudWatch alarm
- Each investigation has a configurable retention period (deleted after expiry)
- The AI may conclude that an event is **not** a threat — ruling something out is a legitimate analytical outcome

HOW AI INVESTIGATIONS WORK

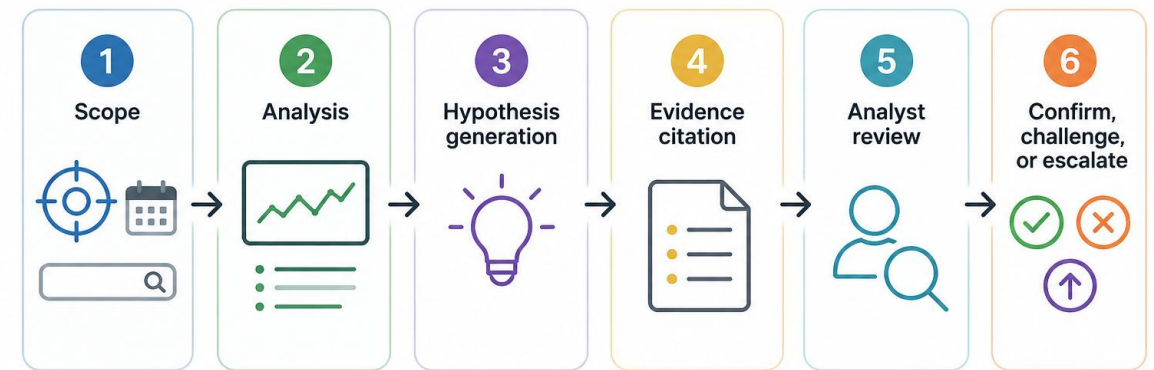
An AI Operations investigation follows a structured pipeline. The analyst sets the scope, the AI runs the analysis, and the analyst reviews the output to make a final call.

The analyst is always the decision-maker. AI provides a starting hypothesis, not a verdict.



HOW AI INVESTIGATIONS WORK

1. Scope — define the log group, time range, and starting query
2. Analysis — AI runs the query, examines the matching events, and evaluates patterns
3. Hypothesis generation — AI produces one or more theories about what the activity represents
4. Evidence citation — each hypothesis cites specific log events supporting the conclusion
5. Analyst review — confirm, challenge, or escalate based on what you know about the environment



SIGNAL VS. NOISE IN LOG DATA

A high-volume production environment can generate tens of millions of log events per day. The challenge for a SOC analyst is not finding logs. It is finding the events that matter within an ocean of routine activity.

AI-assisted investigation helps by grouping similar events, identifying patterns, and surfacing the highest-anomaly items first, reducing the manual filtering burden significantly.



SIGNAL VS. NOISE IN LOG DATA

Noise sources that overwhelm raw log analysis:

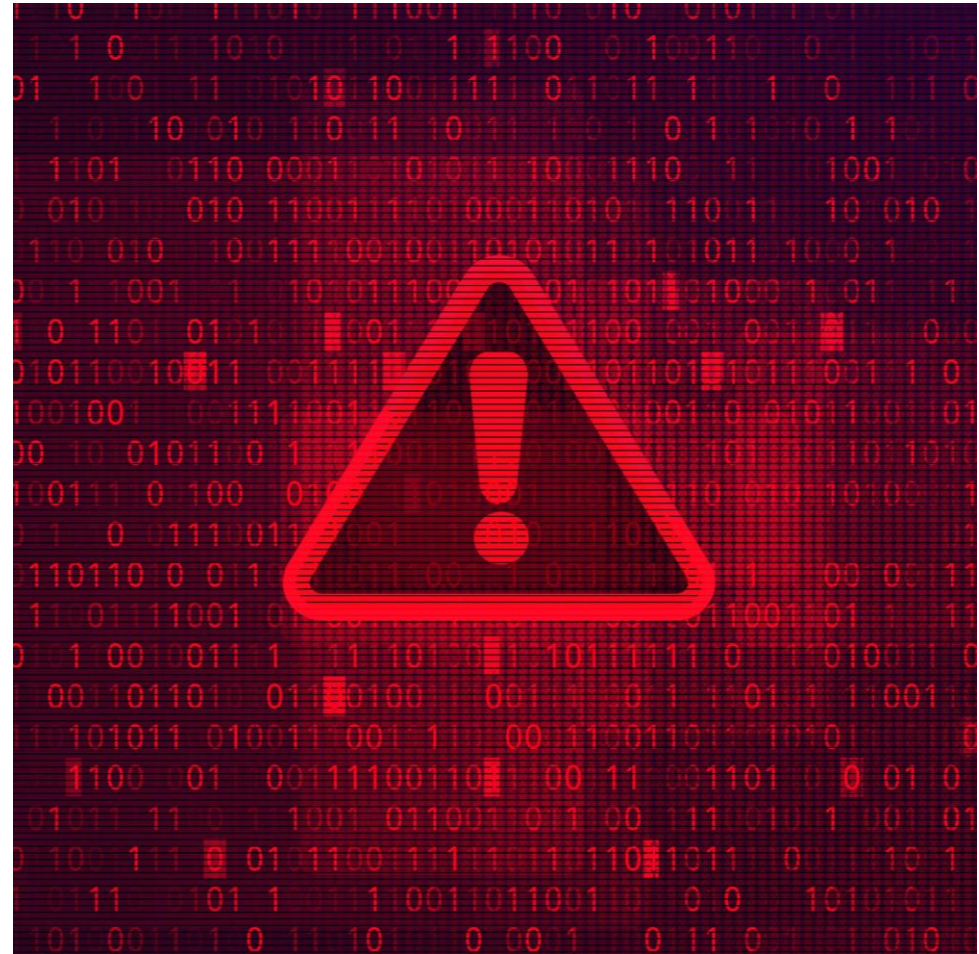
- Scheduled jobs and automated health checks generating repeated identical entries
- Infrastructure-level retries and timeouts that are expected and benign
- Background internet scanner activity probing open ports continuously
- Verbose application debug logging enabled unintentionally in production



MITRE ATT&CK MAPPING IN INVESTIGATIONS

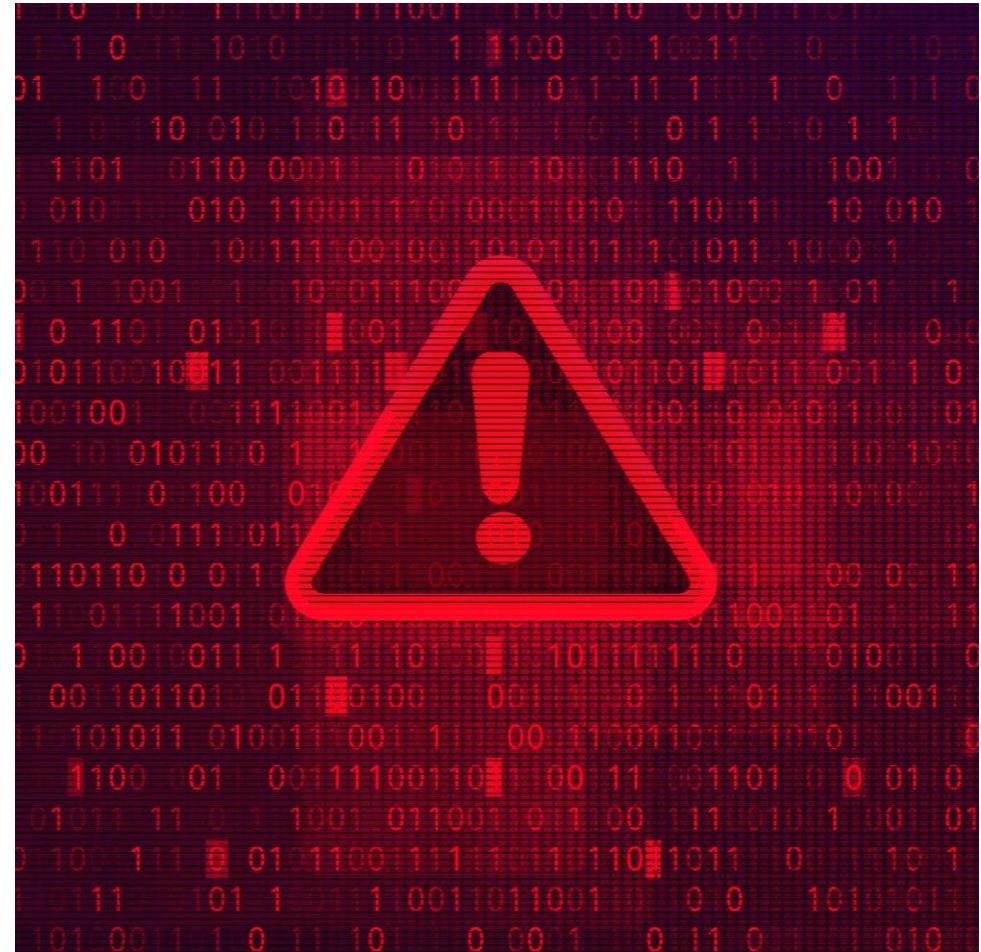
Mapping detected events to ATT&CK techniques gives investigations a structured language and connects findings to known adversary behaviors.

It also helps analysts prioritize. A confirmed T1078 (Valid Accounts abuse) demands different urgency than a routine T1059 (script execution).



MITRE ATT&CK MAPPING IN INVESTIGATIONS

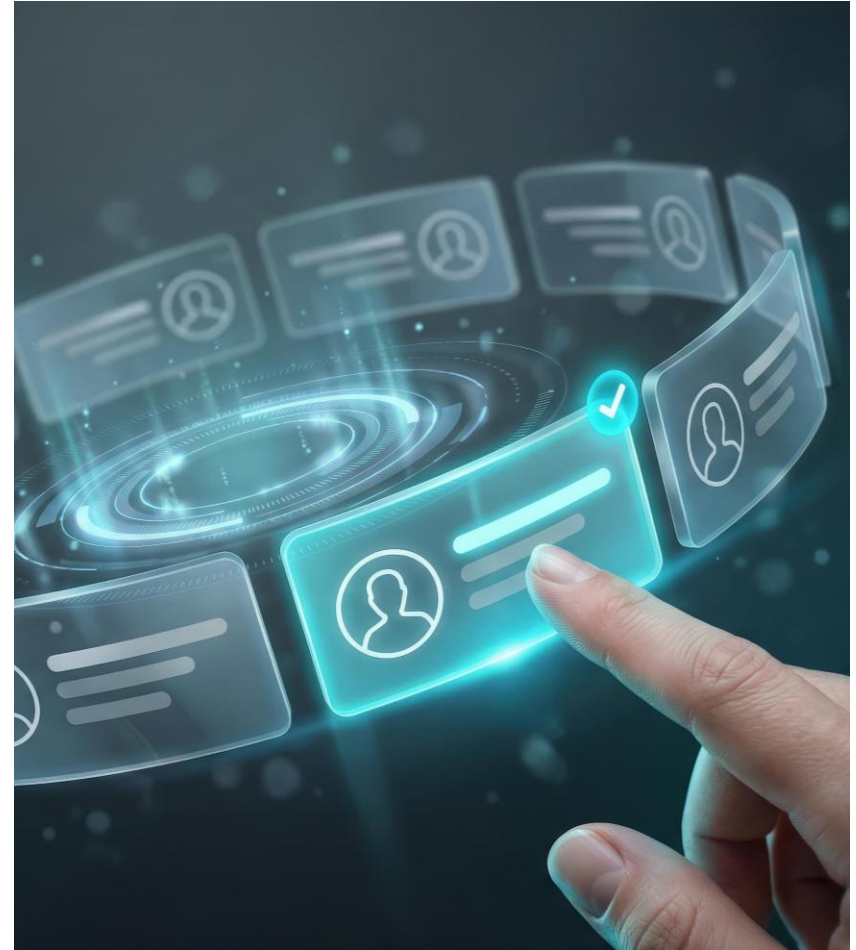
- T1110 — Brute Force: repeated failed SSH login events from a single source
- T1087 — Account Discovery: enumeration commands that appear in audit logs
- T1059 — Command and Scripting Interpreter: shell commands executed on the host
- T1078 — Valid Accounts: authentication with legitimate credentials at unexpected times or locations
- T1526 — Cloud Service Discovery: enumeration of IAM, S3, and EC2 resources via API



IAM ROLES FOR CLOUDWATCH ACCESS

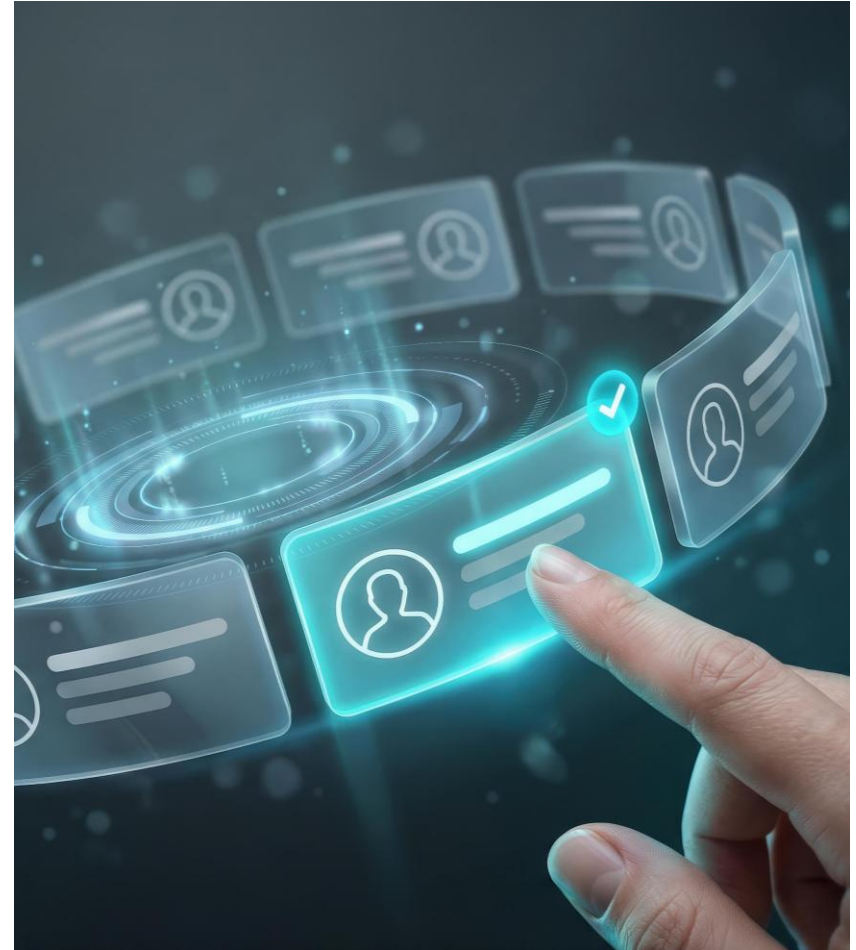
CloudWatch AI Operations and the CloudWatch agent both require IAM permissions to function.

Configuring these permissions correctly is a prerequisite. Without them, the agent cannot ship logs and AI Operations cannot read them.



IAM ROLES FOR CLOUDWATCH ACCESS

- EC2 instance profile — attached to the instance; grants the CloudWatch agent permission to write logs (CloudWatchAgentServerPolicy)
- AI Operations role — an account-level role that CloudWatch assumes to read logs, metrics, and alarms when running an investigation
- Both roles should follow least privilege — the AI Operations role needs read access, not write access, to your resources
- Auto-creating the AI Operations role is the safe default in a lab environment



LOG RETENTION AND INVESTIGATION RETENTION

Logs and investigations stored indefinitely cost money and create data management risk.

Setting appropriate retention periods for both is a standard part of configuring any monitoring environment.



LOG RETENTION AND INVESTIGATION RETENTION

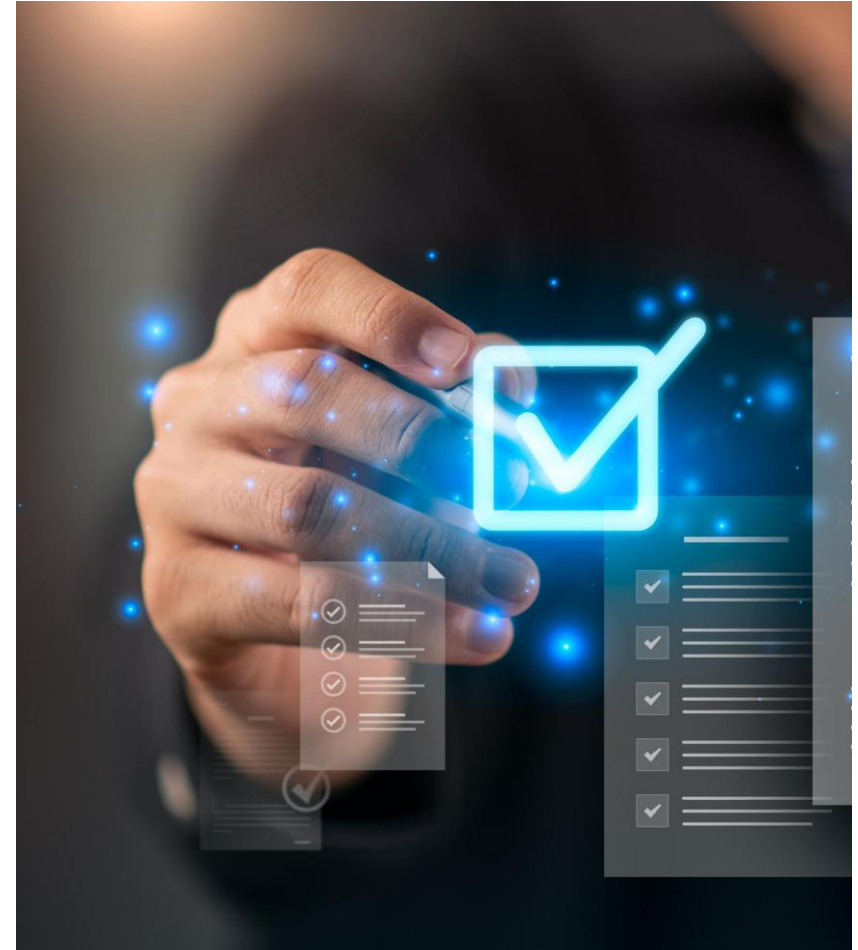
- Default: indefinite (logs are never deleted automatically — you pay for all stored data)
- Configurable per log group: 1 day to 10 years
- Typical SOC practice: 90 days hot (immediately queryable), archive to S3 for long-term compliance retention
- AI Operations investigation retention:
- Configurable at the account level: default 7 days in a lab environment is appropriate
- Expired investigations are deleted automatically — no manual cleanup needed



EVALUATING AI-GENERATED HYPOTHESES

An AI hypothesis is a starting point, not a conclusion. Evaluating it correctly requires the analyst to apply domain knowledge about the environment, the activity that generated the logs, and the plausibility of the AI's interpretation.

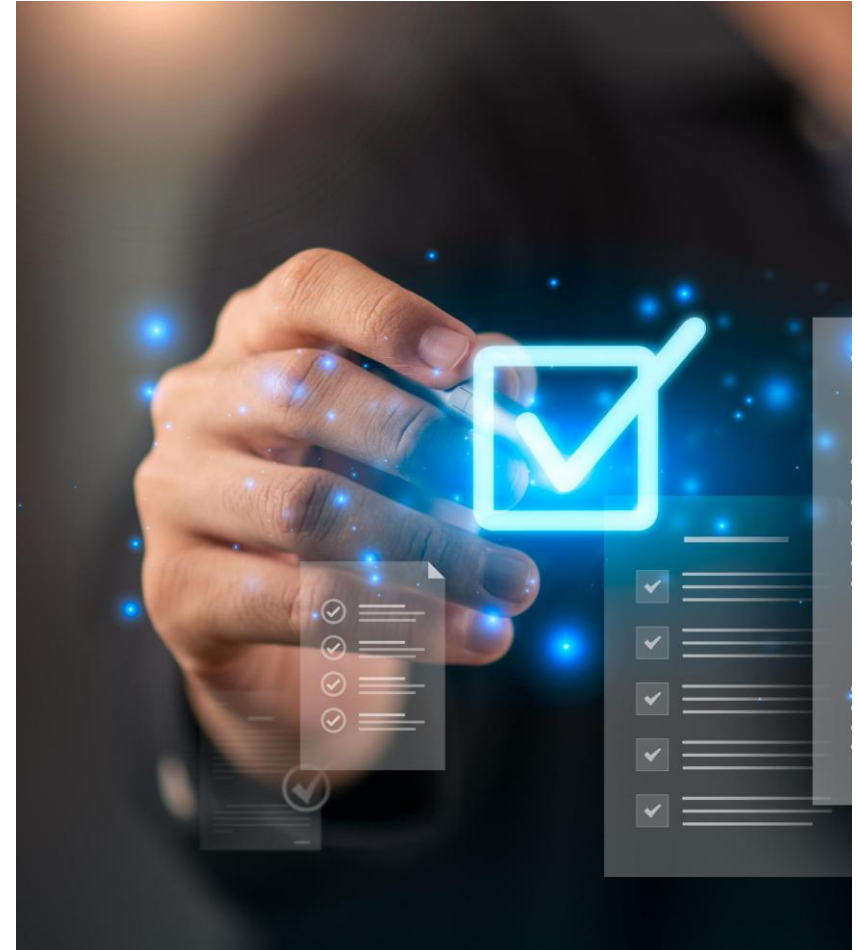
A hypothesis you cannot verify with evidence should not be escalated



EVALUATING AI-GENERATED HYPOTHESES

Questions to ask when reviewing a hypothesis:

- What evidence did the AI cite? Can you locate those specific log events yourself?
- Is the conclusion consistent with the data? Or is the AI pattern-matching on surface features?
- Is the activity expected? A "suspicious" event may be routine if you know what generated it
- What did the AI not consider? Investigations are bounded by the query scope — related signals outside that scope are invisible to the AI



POP QUIZ:

An AI Operations investigation concludes: **"This activity is consistent with a routine automated health check, not an active threat."** How should an analyst respond?

- A. Discard the finding immediately — if the AI says it is safe, no further action is needed
- B. Escalate the finding regardless of the AI's conclusion, because AI tools cannot be trusted to rule out threats
- C. Review the evidence the AI cited, confirm it aligns with your knowledge of expected environment behavior, then close the investigation with a documented rationale
- D. Re-run the investigation with a broader query scope to force a different result



POP QUIZ:

An AI Operations investigation concludes: *"This activity is consistent with a routine automated health check, not an active threat."* How should an analyst respond?

- A. Discard the finding immediately — if the AI says it is safe, no further action is needed
- B. Escalate the finding regardless of the AI's conclusion, because AI tools cannot be trusted to rule out threats
- C. Review the evidence the AI cited, confirm it aligns with your knowledge of expected environment behavior, then close the investigation with a documented rationale**
- D. Re-run the investigation with a broader query scope to force a different result



POP QUIZ:

A student opens CloudWatch and finds the /soc-lab/secure log group is empty. Which of the following is the most likely cause and correct first remediation step?

- A. The log group does not exist yet — create it manually in the CloudWatch console
- B. The CloudWatch agent on the EC2 instance is not running or has lost its configuration — restart it with ``sudo systemctl restart amazon-cloudwatch-agent``
- C. The IAM policy attached to the analyst's user does not include ``logs:GetLogEvents`` — update the policy
- D. CloudWatch Logs Insights requires a 30-minute warm-up period before new log groups become queryable



POP QUIZ:

A student opens CloudWatch and finds the /soc-lab/secure log group is empty. Which of the following is the most likely cause and correct first remediation step?

- A. The log group does not exist yet — create it manually in the CloudWatch console
- B. The CloudWatch agent on the EC2 instance is not running or has lost its configuration — restart it with ``sudo systemctl restart amazon-cloudwatch-agent``**
- C. The IAM policy attached to the analyst's user does not include ``logs:GetLogEvents`` — update the policy
- D. CloudWatch Logs Insights requires a 30-minute warm-up period before new log groups become queryable



LAB 2.1 — CLOUDWATCH AI OPERATIONS



LAB 2.2 — CLOUDWATCH AI OPERATIONS



WHAT IS A SOC DASHBOARD?

A SOC dashboard is a visual workspace that displays the most important security signals in one place, allowing an analyst to assess the current threat picture at a glance without running queries from scratch.

Good dashboards are designed around analyst workflows, not technical capabilities.



WHAT IS A SOC DASHBOARD?

- Actionable signals — every widget should lead to a decision or investigation
- Time-range consistency — all widgets should show the same window for meaningful comparison
- Layered views — high-level counts on top; event detail available on drill-down
- Alert integration — surface active alarms directly on the dashboard, not in a separate tab
- Dashboards that display everything simultaneously are as useless as dashboards that display nothing



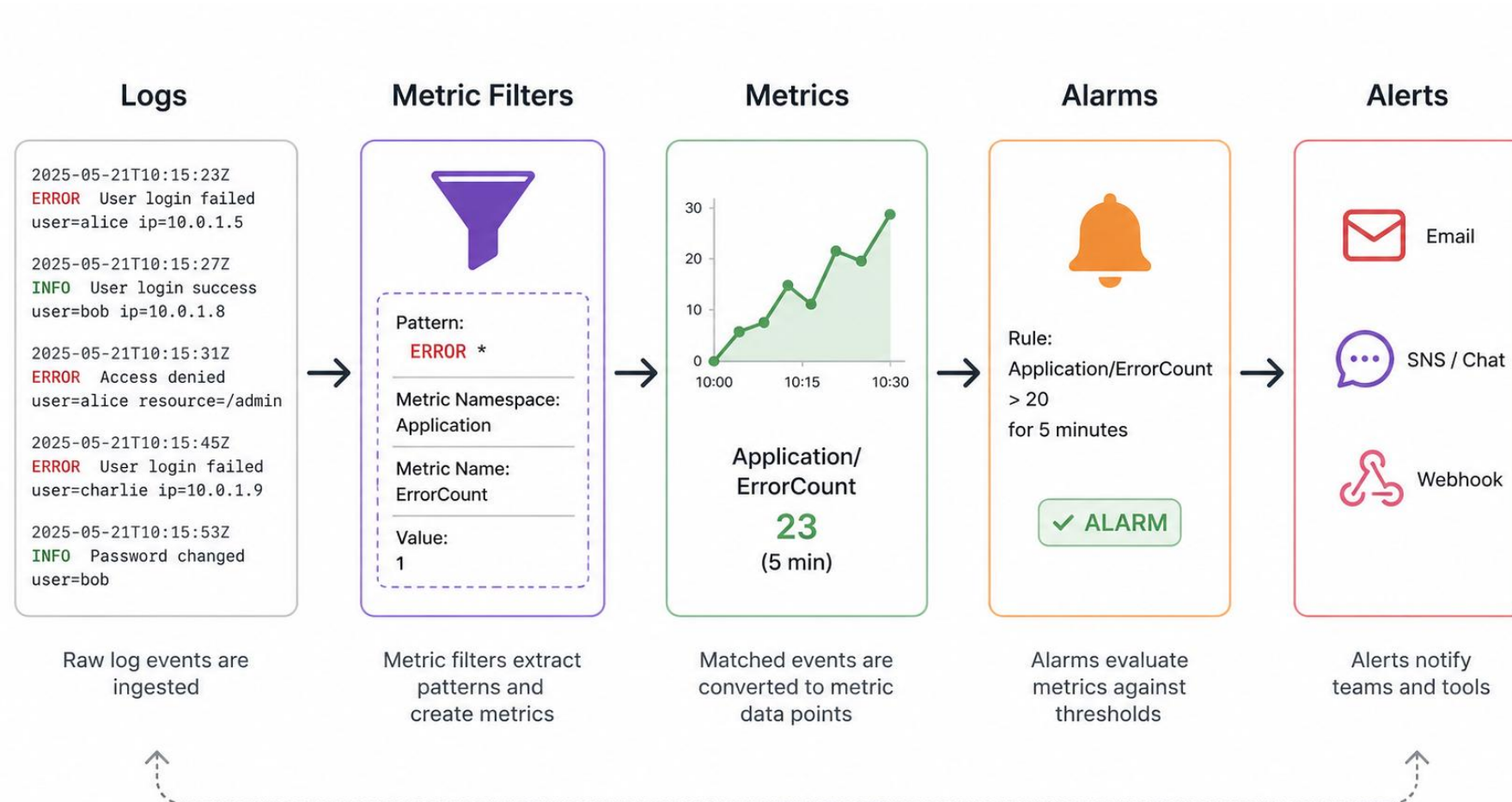
METRIC FILTERS — TURNING LOG EVENTS INTO NUMBERS

A CloudWatch metric filter reads a log group and increments a custom metric counter each time a log entry matches a specified pattern. This converts unstructured log text into a time-series number that can be graphed, alarmed on, and analyzed over time.

Why this matters for detection:

- Raw log events cannot be alarmed on directly, they must first become a metric
- A metric filter creates a countable signal: "how many sudo failures occurred in the last minute?"
- Custom metrics live in a namespace you define (e.g., SOCLab)
- Metric filters are the bridge between log data and the rest of the CloudWatch alerting ecosystem
- One metric filter can feed multiple alarms and dashboard widgets simultaneously

METRIC FILTERS — TURNING LOG EVENTS INTO NUMBERS



WRITING EFFECTIVE METRIC FILTER PATTERNS

Test your pattern against real log data in the Test pattern step of the metric filter wizard before saving.

Pattern syntax:

- Literal string — `exe="/usr/bin/sudo"` matches any log entry containing that exact string
- AND logic — `[a, b]` or space-separated terms: `exe="/usr/bin/sudo" res=failed`
- Wildcard — `%` matches zero or more characters within a term
- Field-based — for JSON logs: `{ $.eventType = "AuthenticationFailure" }`

Create filter pattern

You can use metric filters to monitor events in a log group as they are sent to CloudWatch Logs. You can monitor and count specific terms or extract values from log events and associate the results with a metric. [Learn more about pattern syntax.](#)

Filter pattern

Specify the terms or pattern to match in your log events to create metrics.

☐ Enable metric filter on transformed logs
When enabled, metric filter will be applied to transformed logs. When disabled, metric filter will be applied to original logs.

Field selection criteria - optional Builder Editor

Filtering options based on account and region provided for centralized log groups

Test pattern

The test does not include the field selection criteria

Select log data to test

All log data fetched will be in original log format

Log event messages

Type log data to test with your Filter Pattern. Please use line breaks to separate log events.

```
[83078518-fcc1-4d30-9573-8b9737671438] BENCHMARK : Running
Start Crawl for Crawler TestCrawler2
[83078518-fcc1-4d30-9573-8b9737671438] BENCHMARK :
```


CLOUDWATCH ALARMS — CONFIGURATION AND THRESHOLDS

A CloudWatch alarm monitors a single metric and transitions between states based on whether the metric value crosses a defined threshold.

State transitions can trigger actions notifications, Lambda invocations, or AI Operations investigations.



CLOUDWATCH ALARMS — CONFIGURATION AND THRESHOLDS

Alarm states:

- OK — metric is within the acceptable range
- In alarm — metric has crossed the threshold
- Insufficient data — not enough data points yet to evaluate

Key configuration decisions:

- Statistic — Sum for counting security events; Average for rate or latency metrics
- Period — the evaluation window; 1 minute is appropriate for near-real-time security alerting
- Threshold — ≥ 1 for security events: any single match should trigger the alarm
- Alarm actions — what happens when the alarm fires (the most important field for a SOC pipeline)



ALERT RUNBOOKS & DOCUMENTATION

All alerts should be documented in a centralized wiki with an associated runbook. This ensures consistent response and reduces time to investigate incidents.

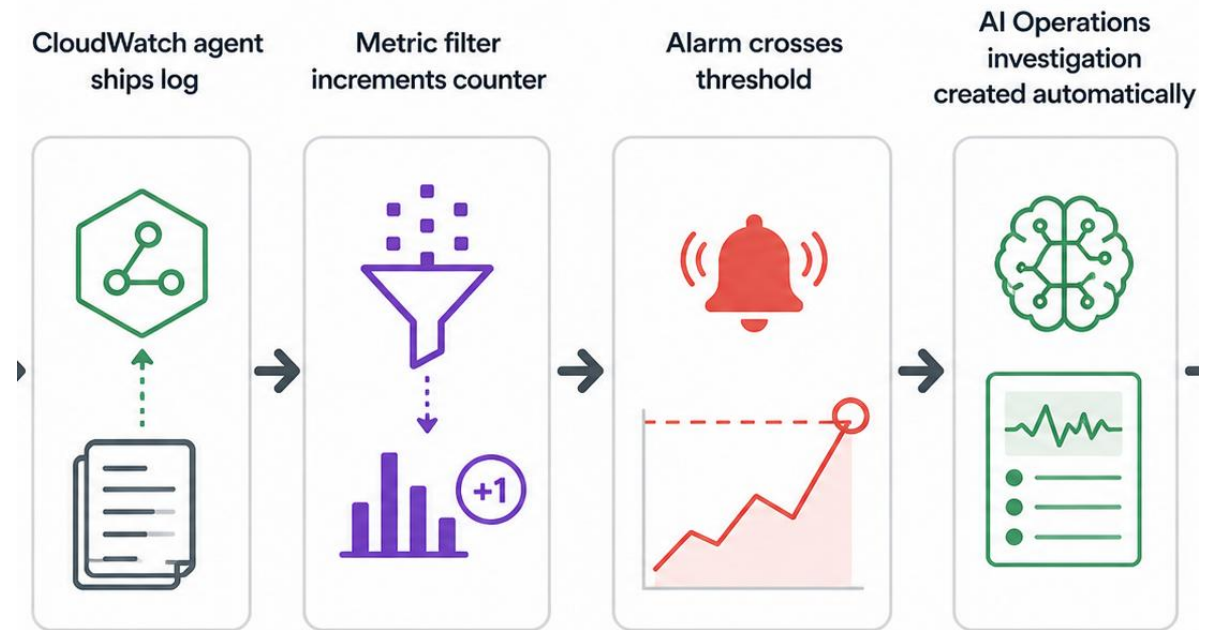
- Each alert links to a runbook
- Where to look: dashboards, logs, metrics
- Suggested remediation steps
- Clear escalation path
- Notes on common causes / patterns



CONNECTING ALARMS TO AI INVESTIGATIONS

Wiring a CloudWatch alarm to an AI Operations investigation creates an automatic triage pipeline:

A suspicious event occurs, the metric increments, the alarm fires, and an investigation starts without analyst intervention. An analyst will review, investigate and act on the results.



PRIVILEGE ESCALATION DETECTION — `SUDO` AND `SU`

Privilege escalation is one of the highest-priority attack categories because success may gives an attacker the same power as a system administrator.

Detecting failed attempts is critical. It reveals that escalation is being actively attempted even before it succeeds.



PRIVILEGE ESCALATION DETECTION — `SUDO` AND `SU`

Detection signals in auditd:

- Failed sudo — exe="/usr/bin/sudo" res=failed in /var/log/audit/audit.log
- Failed su — exe="/usr/bin/su" res=failed
- Sudoers modification — write access to /etc/sudoers or /etc/sudoers.d/
- Unexpected SUID execution — a binary with SetUID bit run by a low-privilege user

MITRE ATT&CK mapping:

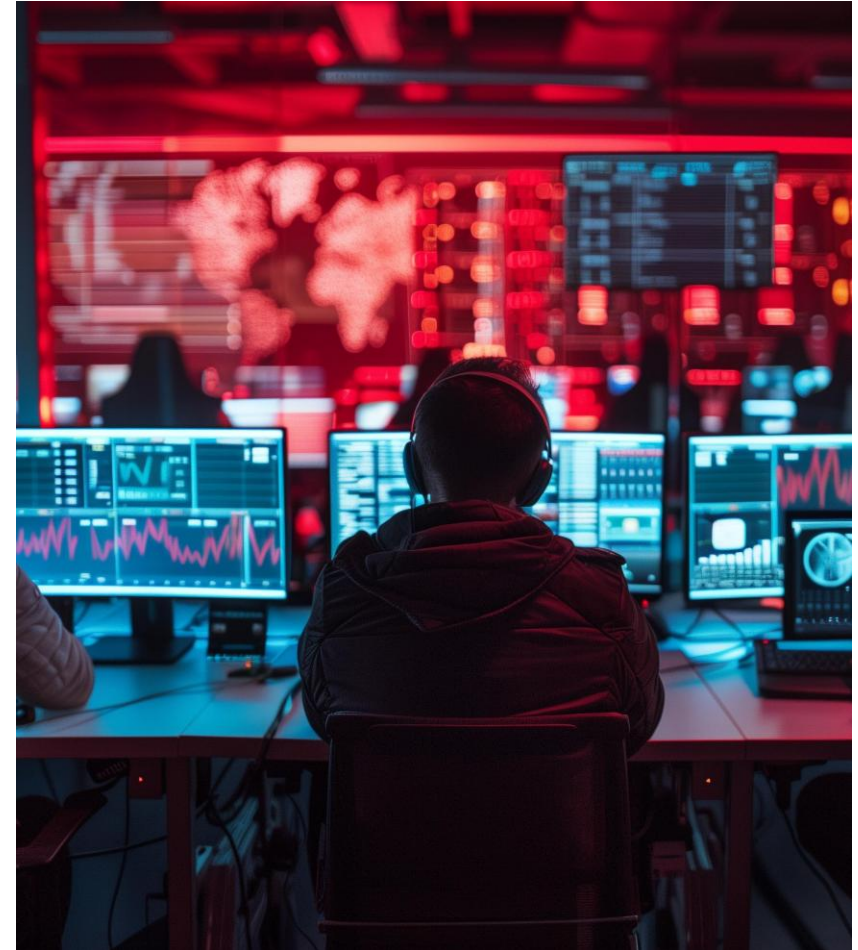
- T1548.003 — Abuse Elevation Control Mechanism: sudo
- T1548.001 — Abuse Elevation Control Mechanism: SetUID



RED AND PURPLE TEAM

Red and Purple Teams are used to test and improve an organization's security posture through simulated attacks and collaboration.

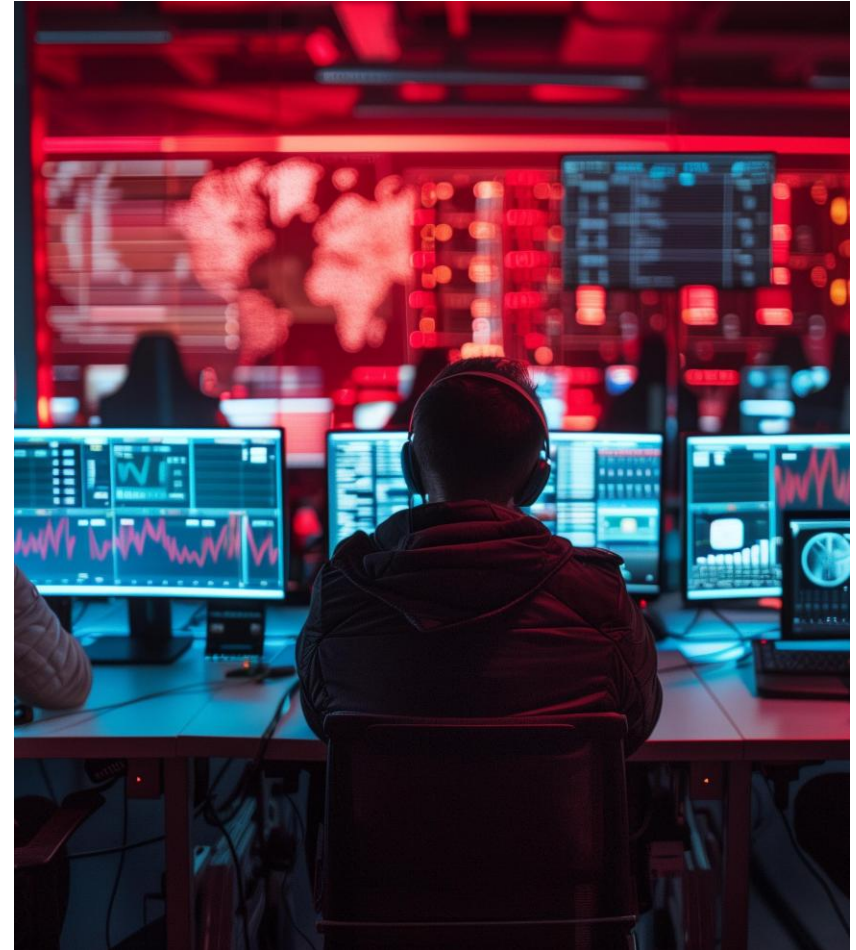
- Red Team: Simulates real attackers (offensive security)
- Focuses on exploitation, evasion, and bypassing controls
- Tests people, processes, and technology
- Purple Team: Collaboration between Red & Blue teams
- Improves detection, alerting, and response
- Focuses on learning and continuous improvement



CREATING A TEST ATTACKER ACCOUNT

To safely simulate attacks without risking the integrity of your lab environment, a dedicated low-privilege test account is used for attack simulation steps.

This mirrors real red team / purple team exercise practices.



DASHBOARD DESIGN FOR SECURITY OPERATIONS

A well-designed SOC dashboard is an analyst's first stop, not their last. It should answer the most common questions immediately without requiring the analyst to build queries from scratch each time they sit down.

Each widget should have a clear, specific Logs Insights query behind it. Not a broad "show me everything" query.



DASHBOARD DESIGN FOR SECURITY OPERATIONS

What to include in a SOC dashboard:

- Event volume over time — a line chart showing total auth events per 5-minute window
- Failed auth by source — a ranked table of IPs with the most failures
- Privilege escalation attempts — a counter or time-series for sudo/su failures
- Active alarms — current alarm state for all security metrics
- Recent raw events — a log table showing the last 20 matching events for quick context



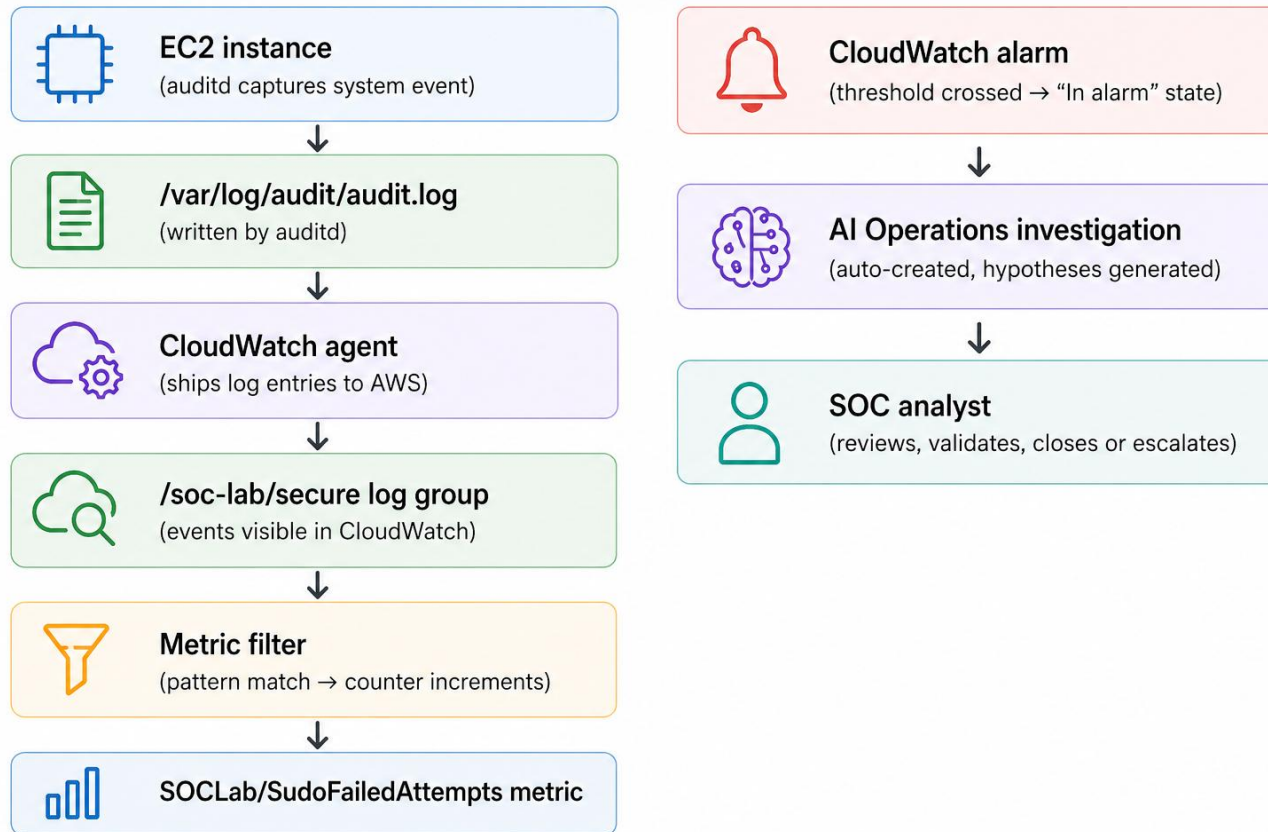
CLOUDWATCH DASHBOARD WIDGETS

CloudWatch dashboards support multiple widget types. Choosing the right type for each signal makes the difference between a dashboard that accelerates decision-making and one that just looks busy.

For a SOC lab dashboard, a combination of a line chart (trend), a logs table (event detail), and an alarm status widget (current alert state) gives an analyst the most useful three-panel view.

Widget Type	Best For
Line chart	Event counts over time; trend detection
Number widget	Current alarm count; total findings today
Logs table	Showing raw recent events with @timestamp and @message
Alarm status	At-a-glance current state of all configured alarms
Bar chart	Comparing counts across discrete categories (e.g., by source IP)

THE FULL DETECTION PIPELINE



Building a detection pipeline means connecting every component from raw host activity through to analyst action.

Understanding the full chain helps you diagnose failures. If an alarm never fires, you need to know which link broke.

ATT&CK TECHNIQUES IN THE LAB ENVIRONMENT

Mapping your detection to ATT&CK validates coverage and identifies gaps.

A few techniques not yet covered (gap):

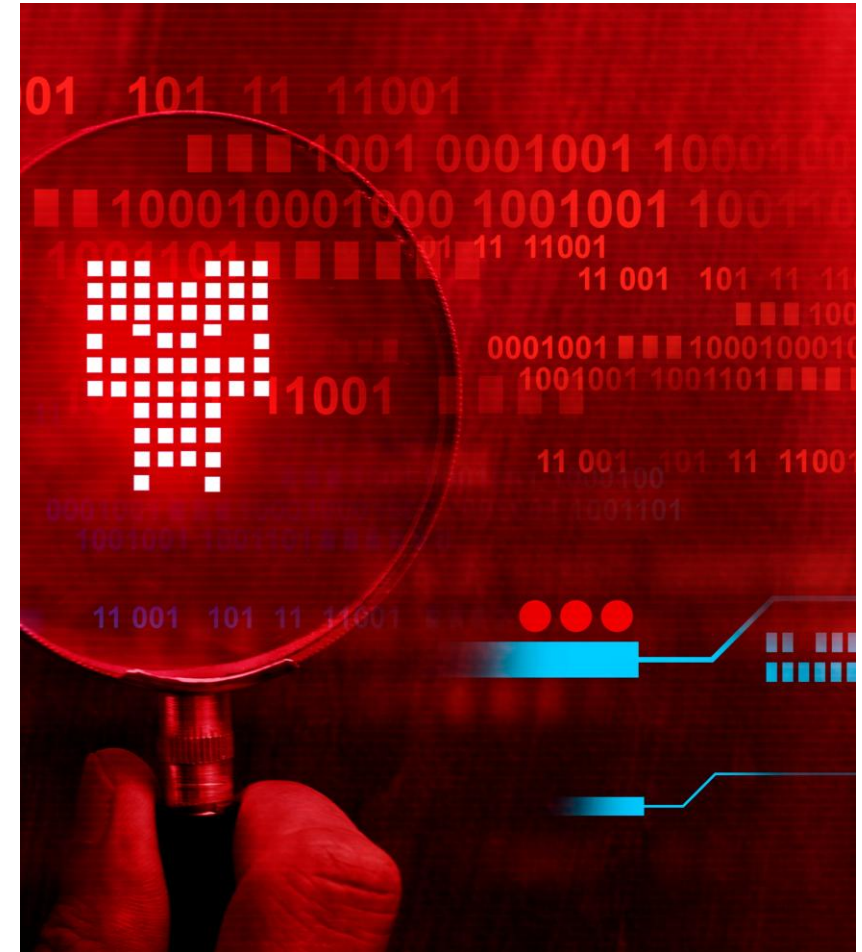
- T1053 — Scheduled Task/Job (cron) → no metric filter exists yet
- T1136 — Create Account → ADD_USER event not yet alerted

Technique	ID	Detection Signal
Brute Force	T1110	Failed SSH auth events
Valid Accounts — abuse	T1078	Auth from unexpected accounts
Abuse Elevation: sudo	T1548.003	exe="/usr/bin/sudo" res=failed
Command and Scripting	T1059	Shell command execution in audit log

TESTING AND VALIDATING DETECTION RULES

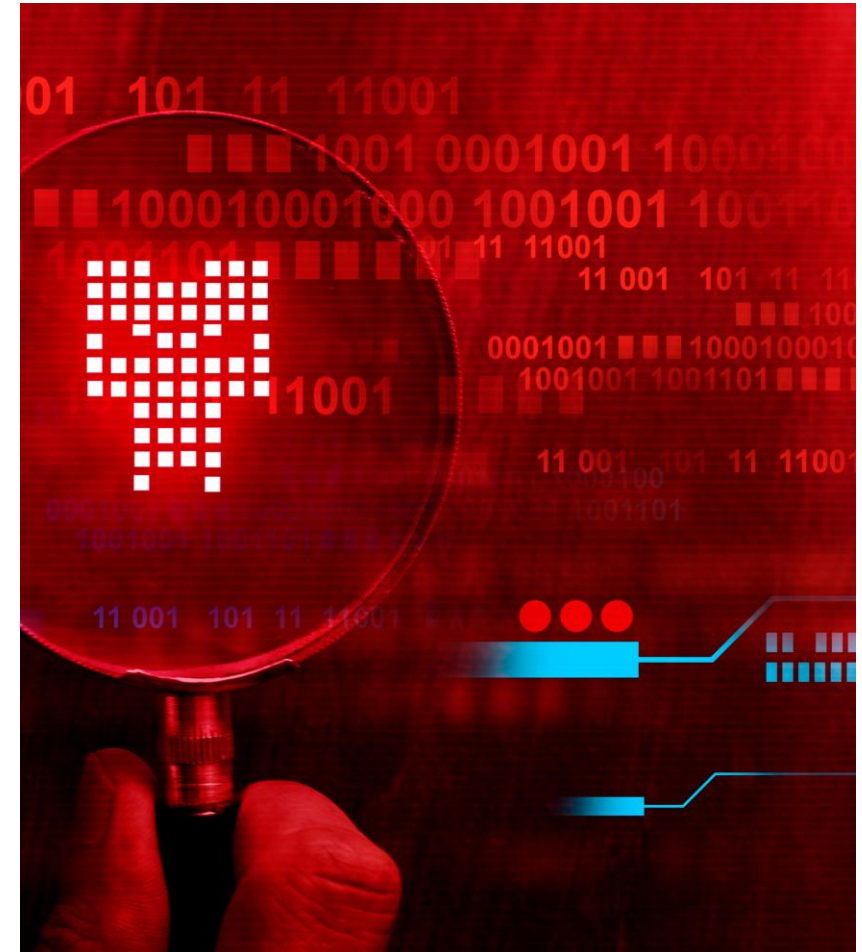
A detection rule that has never fired is not a tested rule, it is a hypothesis.

Every metric filter and alarm must be validated by generating known test events and confirming the pipeline responds as expected.



TESTING AND VALIDATING DETECTION RULES

- Generate test events — run the action the rule should detect (controlled and documented)
- Verify log entry — confirm the expected log line appears in CloudWatch within 60 seconds
- Verify metric increment — confirm the custom metric value increased in the CloudWatch console
- Verify alarm state — confirm the alarm transitions to "In alarm"
- Verify investigation — confirm an AI Operations investigation was automatically created
- Clean up — remove test artifacts (e.g., sudo userdel -r testattacker) after validation



POP QUIZ: You want to create a metric filter that counts failed sudo attempts from the auditd log. Which filter pattern is correct for auditd log entries?

- A. ``Failed sudo attempt``
- B. ``{ $.result = "FAILED" && $.command = "sudo" }``
- C. ``exe="/usr/bin/sudo" res=failed``
- D. ``sudo: authentication failure``



POP QUIZ: You want to create a metric filter that counts failed sudo attempts from the auditd log. Which filter pattern is correct for auditd log entries?

- A. ``Failed sudo attempt``
- B. ``{ $.result = "FAILED" && $.command = "sudo" }``
- C. ``exe="/usr/bin/sudo" res=failed``**
- D. ``sudo: authentication failure``



POP QUIZ: You are configuring a CloudWatch alarm to detect sudo privilege escalation attempts. Which configuration is most appropriate?

- A. Statistic: Average, Period: 1 hour, Threshold: greater than 100
- B. Statistic: Sum, Period: 1 minute, Threshold: greater than or equal to 1
- C. Statistic: Minimum, Period: 5 minutes, Threshold: less than 1
- D. Statistic: Sum, Period: 24 hours, Threshold: greater than 1000



POP QUIZ: You are configuring a CloudWatch alarm to detect sudo privilege escalation attempts. Which configuration is most appropriate?

- A. Statistic: Average, Period: 1 hour, Threshold: greater than 100
- B. Statistic: Sum, Period: 1 minute, Threshold: greater than or equal to 1**
- C. Statistic: Minimum, Period: 5 minutes, Threshold: less than 1
- D. Statistic: Sum, Period: 24 hours, Threshold: greater than 1000



POP QUIZ:

A student generates failed sudo attempts on their EC2 instance but the CloudWatch alarm never fires. Assuming the alarm and metric filter are configured correctly, which component is the most likely point of failure?

- A. The CloudWatch alarm has not been given enough time — alarms take up to 24 hours to activate after creation
- B. The CloudWatch agent on the EC2 instance is not running or is not configured to ship ``/var/log/audit/audit.log``
- C. The metric filter pattern is too specific — it should use a wildcard to match more events
- D. The IAM role attached to the alarm does not have permission to read the audit log file



POP QUIZ: A student generates failed sudo attempts on their EC2 instance but the CloudWatch alarm never fires. Assuming the alarm and metric filter are configured correctly, which component is the most likely point of failure?

- A. The CloudWatch alarm has not been given enough time — alarms take up to 24 hours to activate after creation
- B. The CloudWatch agent on the EC2 instance is not running or is not configured to ship `/var/log/audit/audit.log`**
- C. The metric filter pattern is too specific — it should use a wildcard to match more events
- D. The IAM role attached to the alarm does not have permission to read the audit log file



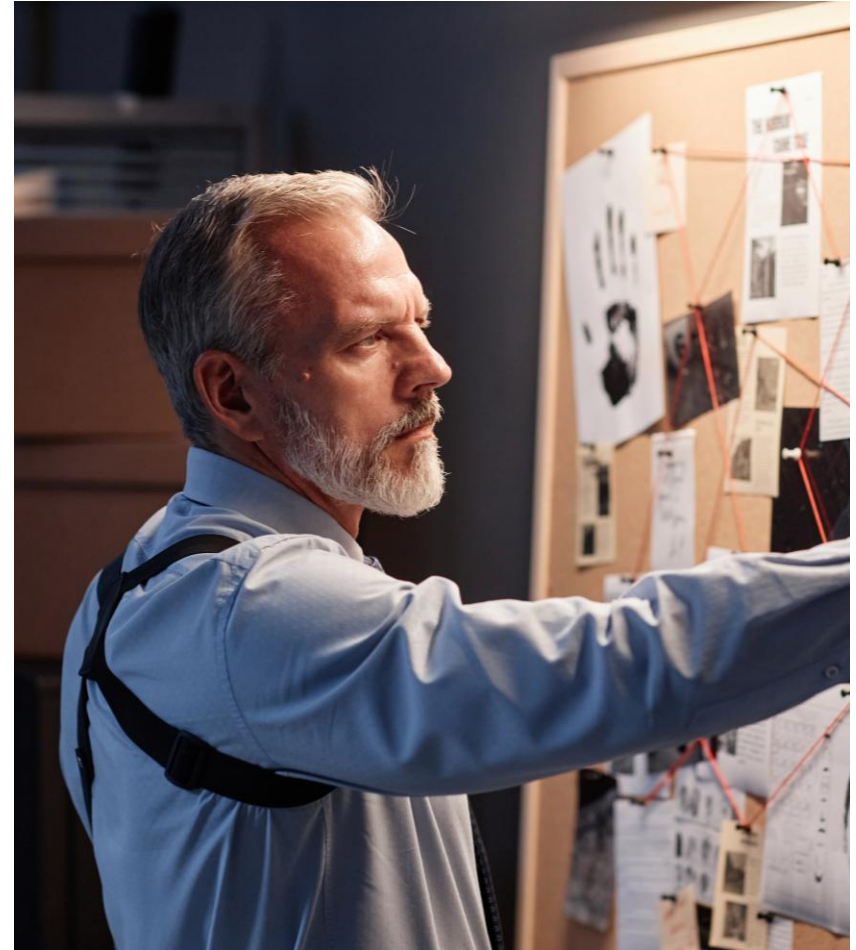
LAB 2.3 — SECURITY MONITORING DASHBOARD AND AUTOMATED DETECTION



DETECTION ENGINEERING — WHAT IT IS

Detection engineering is the discipline of building, testing, and maintaining the rules and pipelines that alert analysts to adversary behavior. It sits at the intersection of threat intelligence, log analysis, and software development.

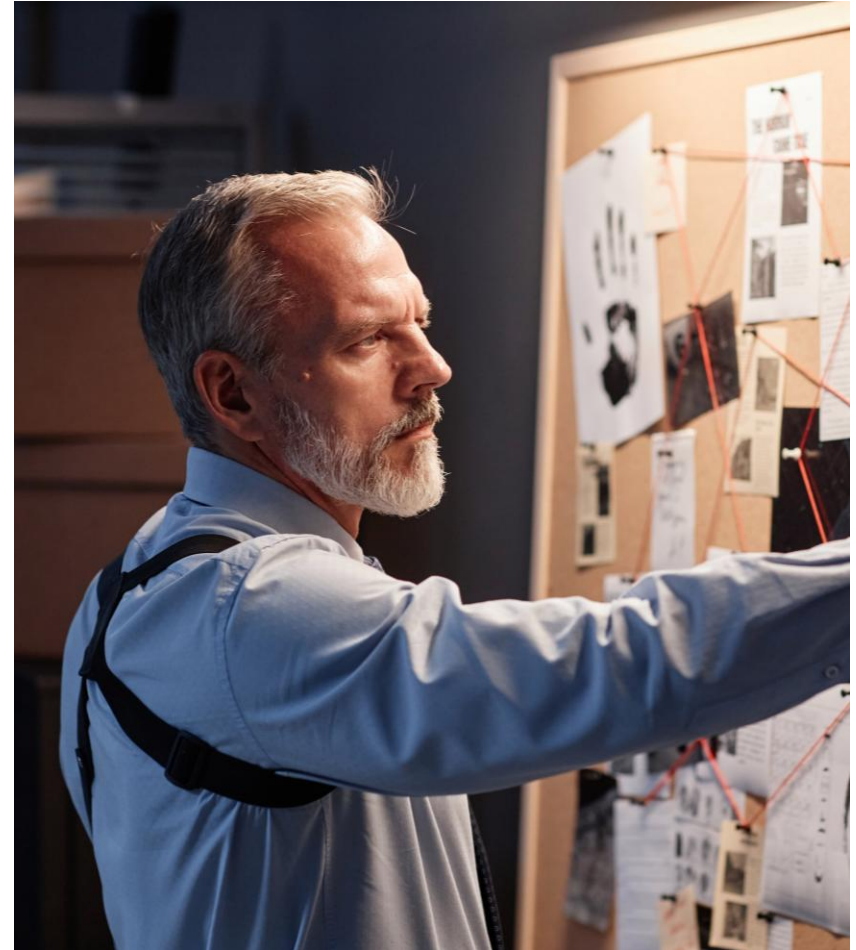
Detection engineering is not a one-time task. It is an ongoing operational discipline.



DETECTION ENGINEERING — WHAT IT IS

A detection engineer:

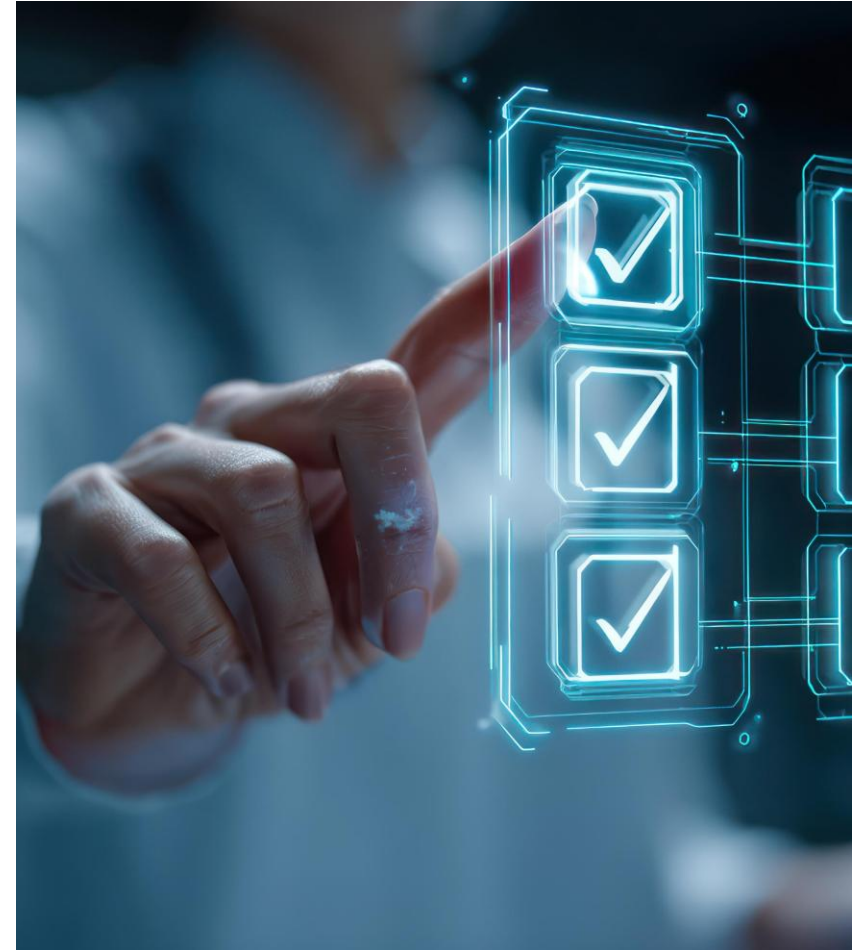
- Translates threat intelligence and ATT&CK techniques into actionable detection logic
- Tests rules against both real attack data and benign activity to calibrate false positive rates
- Maintains and tunes detections as the environment and attacker techniques evolve
- Documents every rule: what it detects, what evidence it needs, and what the analyst should do when it fires



CHOOSING WHAT TO DETECT

Not every possible attacker technique can be detected with the same signal quality. A practical detection strategy starts with the techniques most likely to target your environment and the techniques where you have reliable, high-fidelity log data.

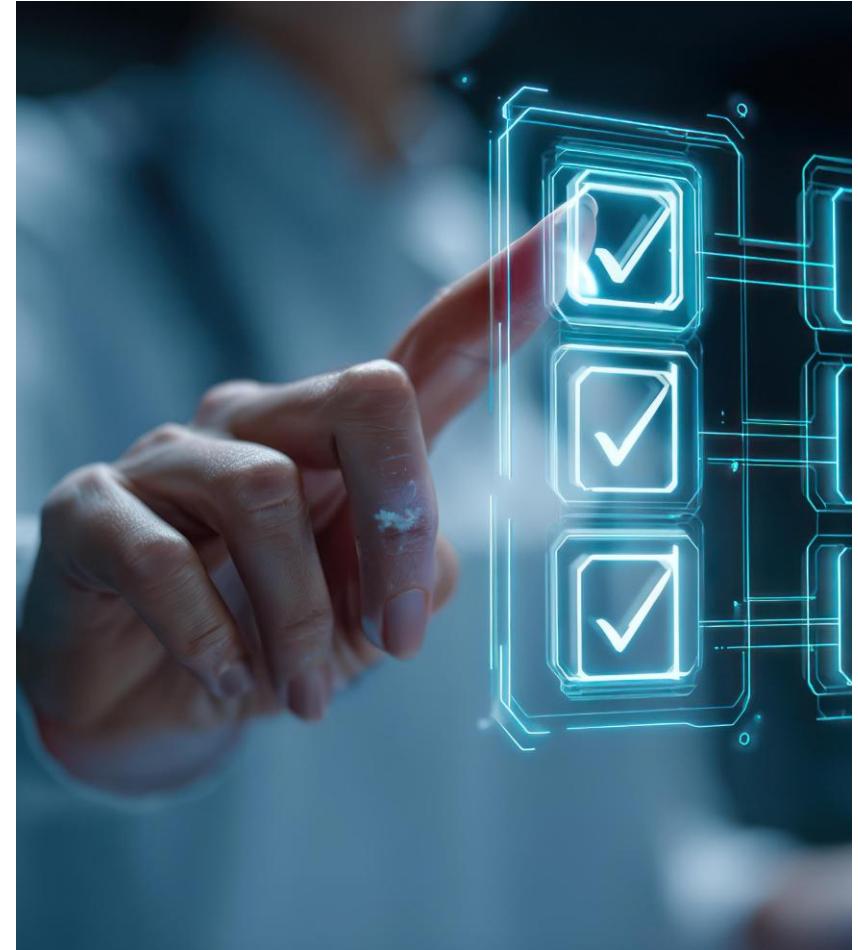
For a Linux EC2 environment with auditd and CloudWatch: SSH brute force, privilege escalation, new account creation, and sensitive file access all score high on all four dimensions.



CHOOSING WHAT TO DETECT

Prioritization framework:

- Signal fidelity — does the technique produce a distinctive, low-noise log event?
- ATT&CK prevalence — how commonly is this technique used in real-world attacks against your type of environment?
- Detection gap — is this technique currently unmonitored?
- Blast radius — if this technique succeeds undetected, how severe is the impact?



LINUX ENDPOINT THREAT CATALOG

Five specific threat techniques are relevant to the lab environment.

Each produces a distinctive auditd log signature and maps to a documented ATT&CK technique.

Threat	What Happens	auditd Signal	ATT&CK
SSH brute force	Repeated failed SSH logins from one source	exe="/usr/sbin/sshd" res=failed	T1110
New account backdoor	useradd creates a persistence account	type=ADD_USER	T1136
Sensitive file access	Read attempt on /etc/shadow	name="/etc/shadow"	T1003
su abuse	Repeated failed su - root attempts	exe="/usr/bin/su" res=failed	T1548
Cron persistence	Cron job installed by non-root user	crontab keyword in audit log	T1053

AUDITD — THE LINUX AUDIT DAEMON

```
[ec2-user@ip-172-31-34-219 ~]$ sudo auditctl -w /tmp/audit-demo.txt -p rwx -k demo_watch
Old style watch rules are slower
[ec2-user@ip-172-31-34-219 ~]$ cat /tmp/audit-demo.txt
echo "new line" >> /tmp/audit-demo.txt
hello
[ec2-user@ip-172-31-34-219 ~]$ sudo ausearch -k demo_watch -i
---
type=PROCTITLE msg=audit(04/30/26 02:15:47.083:16354) : proctitle=cat /tmp/audit-demo.txt
type=PATH msg=audit(04/30/26 02:15:47.083:16354) : item=0 name=/tmp/audit-demo.txt
inode=3352 dev=00:1f mode=file,644 ouid=ec2-user ogid=ec2-user rdev=00:00
obj=unconfined_u:object_r:user_tmp_t:s0 nametype=NORMAL cap_fp=none cap_fi=none cap_fe=0
cap_fver=0 cap_frootid=0
...
```

^^ Audit reads to a file

auditd is the kernel-level auditing system for Linux. It records system calls, file accesses, and process execution events at a level of detail no userspace logging tool can match.

Since it runs in the kernel, it is extremely difficult for a user-level attacker to suppress without root access.

AUDITD — THE LINUX AUDIT DAEMON

```
[ec2-user@ip-172-31-34-219 ~]$ echo "test" > /tmp/audit-chmod.txt
[ec2-user@ip-172-31-34-219 ~]$ sudo auditctl -a always,exit -F arch=b64 -S \
chmod,fchmod,fchmodat -k chmod_watch

[ec2-user@ip-172-31-34-219 ~]$ chmod 777 /tmp/audit-chmod.txt
[ec2-user@ip-172-31-34-219 ~]$ sudo ausearch -k chmod_watch -i
----
type=PROCTITLE msg=audit(04/30/26 04:16:57.050:16660) : proctitle=chmod 777 /tmp/audit-
chmod.txt
type=PATH msg=audit(04/30/26 04:16:57.050:16660) : item=0 name=/tmp/audit-chmod.txt
inode=3491 dev=00:1f mode=file,644 ouid=ec2-user ogid=ec2-user rdev=00:00
obj=unconfined_u:object_r:user_tmp_t:s0 nametype=NORMAL cap_fp=none cap-fi=none cap-fe=0
cap_fver=0 cap_frootid=0
type=CWD msg=audit(04/30/26 04:16:57.050:16660) : cwd=/home/ec2-user
type=SYSCALL msg=audit(04/30/26 04:16:57.050:16660) : arch=x86_64 syscall=fchmodat
success=yes exit=0 a0=AT_FDCWD a1=0x55a40dcf2c50 a2=0777 a3=0x0 items=1 ppid=99309
pid=99402 auid=ec2-user uid=ec2-user gid=ec2-user euid=ec2-user suid=ec2-user fsuid=ec2-
user egid=ec2-user sgid=ec2-user fsgid=ec2-user tty=pts0 ses=19 comm=chmod
exe=/usr/bin/chmod subj=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
key=chmod_watch
```

^^ Audit “chmod” and related commands

- Audit rules — configured with auditctl; define what events to record
- Keys — labels attached to rules (e.g., -k shadow-access) for filtering with ausearch
- /var/log/audit/audit.log — the primary output file; plain-text key-value format
- auditd must be installed and running: sudo systemctl enable --now auditd
- The CloudWatch agent ships this file to the /soc-lab/secure log group

FILE PERMISSION & ACCESS MONITORING (ATT&CK MAPPING)

Monitoring file permission changes and access events helps detect privilege escalation and credential theft. Changes like setUID/setGID and sensitive file reads are strong indicators of malicious activity.

chmod Monitoring (setUID / setGID)

- Detects privilege escalation via modified binaries
- Example: `chmod u+s /usr/bin/bash`
- Maps to: MITRE ATT&CK **T1548 – Abuse Elevation Control Mechanism**

Sensitive File Access Monitoring

- Tracks reads of `/etc/passwd`, `/etc/shadow`, SSH keys
- Detects credential harvesting activity
- Maps to: MITRE ATT&CK **T1003 – OS Credential Dumping**



auditd Logs (chmod example)

```
type=PROCTITLE msg=audit(04/30/26 04:16:57.050:16660) :  
proctitle=chmod 777 /tmp/audit-chmod.txt
```

```
type=PATH msg=audit(04/30/26 04:16:57.050:16660) :  
item=0 name=/tmp/audit-chmod.txt inode=3491 dev=00:1f  
mode=file,644 ouid=ec2-user ogid=ec2-user rdev=00:00  
obj=unconfined_u:object_r:user_tmp_t:s0 nametype=NORMAL ...
```

```
type=CWD msg=audit(04/30/26 04:16:57.050:16660) :  
cwd=/home/ec2-user
```

```
type=SYSCALL msg=audit(04/30/26 04:16:57.050:16660) :  
arch=x86_64 syscall=fchmodat success=yes exit=0  
a0=AT_FDCWD a1=0x55a40dcf2c50 a2=0777 a3=0x0 items=1  
ppid=99309 pid=99402 auid=ec2-user uid=ec2-user  
gid=ec2-user euid=ec2-user suid=ec2-user fsuid=ec2-user  
egid=ec2-user sgid=ec2-user fsgid=ec2-user tty=pts0  
ses=19 comm=chmod exe=/usr/bin/chmod  
subj=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023  
key=chmod_watch
```



Metric Filter

Filter pattern

```
{{$.type = "SYSCALL"}} &&  
{$.syscall = "fchmodat"}} &&  
{$.a2 = "0777"}} &&  
{$.exe = "/usr/bin/chmod"}}  
}
```

Metric namespace

SOCLab

Metric name

Chmod777Count

Value

1

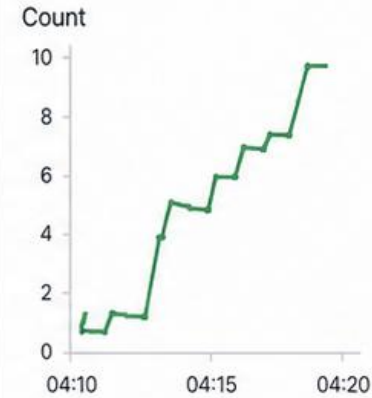
Unit

Count



CloudWatch Metric

SOCLab/Chmod777Count



9

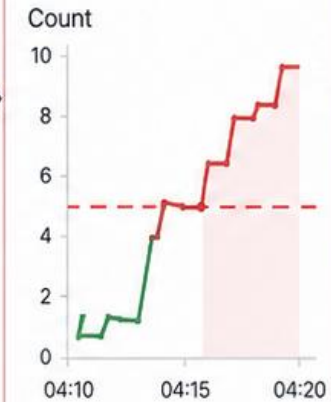
(5 min total)



CloudWatch Alarm

When

Chmod777Count >= 5
for 5 minutes



In alarm



AI Operations Investigation

(auto-created,
hypotheses generated)



SOC Analyst

(reviews, validates,
closes or escalates)

DESIGNING A DETECTION — FROM TECHNIQUE TO RULE

Building a detection for a specific ATT&CK technique requires working backwards from the observable behavior to the log event it produces, then writing the filter that matches it precisely.



DESIGNING A DETECTION — FROM TECHNIQUE TO RULE

Example: detecting su abuse (T1548)

1. Technique: attacker runs `su - root` with a wrong password to attempt privilege escalation
2. System behavior: `su` calls PAM, PAM fails, `auditd` records the failure
3. Log entry: `type=USER_AUTH msg=audit(...): exe="/usr/bin/su" res=failed`
4. Filter pattern: `exe="/usr/bin/su" res=failed`
5. Metric: SuAbuse in the SOCLab namespace
6. Alarm: ≥ 1 occurrences in 1 minute



SIMULATING ATTACKS SAFELY



Attack simulation in a lab is controlled, documented, and scoped to an isolated environment. The goal is to generate the same log events a real attacker would produce, without doing any actual damage.

This approach mirrors a purple team exercise: the attacker and defender role are played by the same person, with full visibility into both sides.

SIMULATING ATTACKS SAFELY



- **Run attack commands only on your own lab instance, never against any other system**
- Use the testattacker account for steps that require a non-privileged user
- Steps requiring sudo must be run as ec2-user — testattacker has no sudo
- Document what you ran, when, and what log events it produced
- Clean up test artifacts after validation: remove test accounts, cron entries, and audit rules

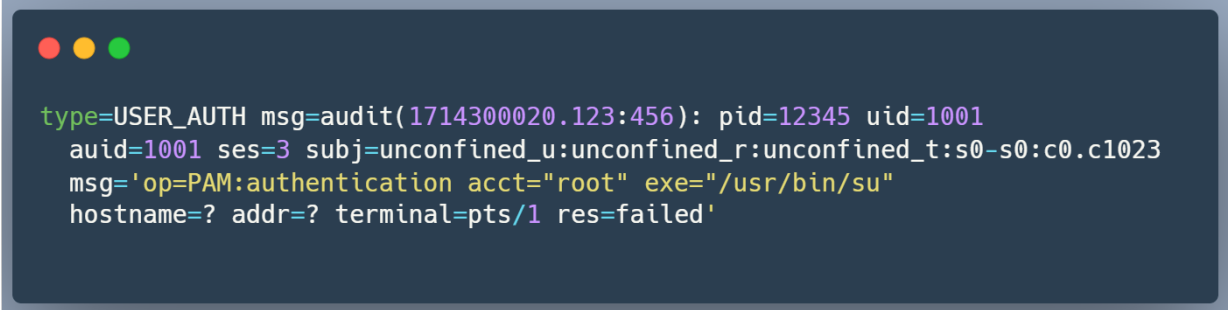
READING AUDITD LOG ENTRIES



```
type=USER_AUTH msg=audit(1714300020.123:456): pid=12345 uid=1001
  auid=1001 ses=3 subj=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
  msg='op=PAM:authentication acct="root" exe="/usr/bin/su"
  hostname=? addr=? terminal=pts/1 res=failed'
```

auditd log entries are dense but follow a consistent key-value format. Learning to read them is a prerequisite for writing accurate filter patterns and validating that your detection rules fire on the right events.

READING AUDITD LOG ENTRIES



```
type=USER_AUTH msg=audit(1714300020.123:456): pid=12345 uid=1001
aid=1001 ses=3 subj=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
msg='op=PAM:authentication acct="root" exe="/usr/bin/su"
hostname=? addr=? terminal=pts/1 res=failed'
```

Example entry — failed su attempt:

Key fields:

- exe — the binary that triggered the event
- res — success or failed
- acct — the account being targeted
- aid — the audit UID of the user who initiated the action
- type — the event category (USER_AUTH, ADD_USER, SYSCALL, etc.)

INCIDENT REPORTING — STRUCTURE AND PURPOSE

An incident report is the formal record of a security event from initial detection through closure.

It serves as documentation for the current event, an artifact for compliance, and a learning record for improving future detections.



INCIDENT REPORTING — STRUCTURE AND PURPOSE

- Incident summary — one paragraph: what happened, when, and how it was detected
- Timeline — ordered list of key events with UTC timestamps
- Attack details — which technique, which account or source, what was targeted
- Detection details — which metric filter and alarm fired; what the AI investigation concluded
- Analyst assessment — do you agree with the AI hypothesis? What did you verify?
- Closure — confirmed mitigated, confirmed false positive, or escalated for further action





Incident Report

Incident ID: INC-2026-000123 | Status: Open



Reported
Apr 30, 2026 04:23:12 UTC



Severity
Medium



Category
Privilege Misuse



Reported By
CloudWatch Alarm



Source
CloudWatch / auditd



Summary

chmod 777 was executed on /tmp/audit-chmod.txt on EC2 instance i-0abc123def456. Detected via auditd logs and CloudWatch metric.



Timeline

- 04:16:57 **chmod 777 executed**
auditd captured event (syscall=fchmodat)
- 04:17:01 **Metric filter matched**
Chmod777Count incremented
- 04:17:01 **Alarm threshold crossed**
Chmod777Count >= 5 for 5 minutes
- 04:17:02 **AI Operations investigation created**
Hypotheses generated
- 04:20:15 **Analyst acknowledged**
Investigation opened



Affected Resources

Resource ID	Type	Details
i-0abc123def456	EC2 Instance	auditd log source
/tmp/audit-chmod.txt	File	Permissions changed to 0777



Detection

Detection Method	CloudWatch Alarm
Rule / Metric	Chmod777Count >= 5 for 5 minutes

Key Evidence (excerpt)

```
type=PROCTITLE msg=audit(04/30/26 04:16:57.050:16660) : proctitle=chmod 777 /tmp/audit-chmod.txt
type=SYSCALL msg=audit(04/30/26 04:16:57.050:16660) : arch=x86_64 syscall=fchmodat success=yes
a2=0777 ... comm=chmod exe=/usr/bin/chmod key=chmod_watch
```



Investigation (AI Operations)

Investigation ID: inv-2026-000123

Hypothesis	Confidence	Summary
Unauthorized permission change	High	chmod 777 on /tmp/audit-chmod.txt by ec2-user
Potential persistence setup	Medium	World-writable file in /tmp could be abused
Legitimate activity	Low	Possible testing or approved maintenance



Impact

Impact Scope	Instance
Data Sensitivity	Low
Business Impact	Low
Availability Impact	None



Next Steps

- ☐ Validate change with system owner
- ☐ Review related activity around the event
- ☐ Determine if file should be removed or remediated
- ☐ Close or escalate as needed



Analyst Review

Analyst	Notes	Disposition	Resolved At
Enter name	Enter notes	Select disposition	—

AI HYPOTHESIS TRIAGE

An AI Operations investigation may generate multiple competing hypotheses for the same set of events. Deciding which is most credible requires applying analyst judgment to the AI's evidence citations.

- Read all hypotheses before accepting any. AI may have found something more important than the first item listed
- Locate the cited evidence in the raw log data
- Apply environment context. Is this activity expected given what you know about the system?
- Identify the most specific hypothesis. A hypothesis that cites a specific account, IP, and technique is more actionable than a vague "unusual activity" conclusion
- Document your decision, even if you close the investigation as a false positive



DETECTION COVERAGE AND GAPS

Building one detection rule is the start, not the end. A mature detection program maps every monitored technique against the full ATT&CK matrix to understand what is covered, what is partial, and what is a complete blind spot.

For the lab environment after Lab 2.4: you will have coverage for 3-5 techniques. Identifying the remaining gaps and explaining what data source and rule would close each one is a core detection engineering skill.



DETECTION COVERAGE AND GAPS

- Which ATT&CK techniques affect my environment and have no detection rule at all?
- Which detections exist but have never been tested with real attack data?
- Which detections fire frequently but rarely result in confirmed true positives (noise)?
- Which techniques require data sources that are not yet collected?



CLOSING AN INVESTIGATION — DECISION FRAMEWORK

Every investigation must be formally closed with a documented decision. Leaving investigations open or marking everything as "resolved" without review is as bad as having no detection in the first place.

Always document **why** you made the decision, not just what it was. This record is the input for tuning the detection rule to reduce future false positives.

Decision	Meaning	Required documentation
True Positive	Real attack activity confirmed	Timeline, technique, impact, remediation taken
False Positive	Expected or benign activity	Why the alert fired, why it is not a threat
Inconclusive	Evidence is ambiguous	What was checked, what additional data is needed
Escalated	Requires higher-tier analyst	What was found, why escalation is needed

POP QUIZ: You want to detect read attempts against /etc/shadow using a CloudWatch metric filter on the auditd log. Which filter pattern is correct?

- A. ``shadow file read detected``
- B. ``name="/etc/shadow"``
- C. ``{ $.file = "/etc/shadow" && $.action = "read" }``
- D. ``cat /etc/shadow``



POP QUIZ: You want to detect read attempts against /etc/shadow using a CloudWatch metric filter on the auditd log. Which filter pattern is correct?

- A. ``shadow file read detected``
- B. ``name="/etc/shadow"``**
- C. ``{ $.file = "/etc/shadow" && $.action = "read" }``
- D. ``cat /etc/shadow``



POP QUIZ: A detection engineer wants to build a rule for T1136. Create Account (an attacker runs useradd to create a backdoor persistence account). Which approach correctly implements this detection?

- A. Create a metric filter on the sshd log group for the pattern `new user created`
- B. Create a metric filter on `/soc-lab/secure` for the auditd pattern `type=ADD\_USER`, then create an alarm triggering on `>= 1` in 1 minute
- C. Set a CloudWatch alarm on the EC2 `CPUUtilization` metric, which spikes when `useradd` runs
- D. Install a third-party agent on the EC2 instance — auditd cannot detect account creation events



POP QUIZ:

A detection engineer wants to build a rule for T1136 — Create Account (an attacker runs `useradd` to create a backdoor persistence account). Which approach correctly implements this detection?

- A. Create a metric filter on the `sshd` log group for the pattern ``new user created``
- B. Create a metric filter on ``/soc-lab/secure`` for the `auditd` pattern ``type=ADD_USER``, then create an alarm triggering on ``>= 1`` in 1 minute**
- C. Set a CloudWatch alarm on the EC2 ``CPUUtilization`` metric, which spikes when ``useradd`` runs
- D. Install a third-party agent on the EC2 instance — `auditd` cannot detect account creation events



POP QUIZ:

An AI Operations investigation was automatically triggered by a sudo failure alarm. The investigation concludes the failed attempt was made by ec2-user running a mistyped command. You confirm this by reviewing the raw audit log. What is the correct closure decision and documentation?

- A. True Positive — escalate to Tier 2 for further investigation of the `ec2-user` account
- B. False Positive — document that the alarm fired on an expected user running a mistyped command, and consider tuning the alarm to exclude `ec2-user`
- C. Inconclusive — close without documentation since it was clearly benign
- D. Escalated — all `sudo` failures involving `ec2-user` must be sent to the security team regardless of context



POP QUIZ:

An AI Operations investigation was automatically triggered by a sudo failure alarm. The investigation concludes the failed attempt was made by ec2-user running a mistyped command. You confirm this by reviewing the raw audit log. What is the correct closure decision and documentation?

- A. True Positive — escalate to Tier 2 for further investigation of the `ec2-user` account
- B. False Positive — document that the alarm fired on an expected user running a mistyped command, and consider tuning the alarm to exclude `ec2-user`**
- C. Inconclusive — close without documentation since it was clearly benign
- D. Escalated — all `sudo` failures involving `ec2-user` must be sent to the security team regardless of context



LAB 2.4 — ATTACK AND DETECT CAPSTONE



CONGRATULATIONS

Over two days you built a complete, working security detection and investigation pipeline on AWS with an automated AI-assisted investigations triggered by real attack simulations.

The skills you practiced here, telemetry configuration, detection engineering, AI hypothesis evaluation, and structured incident closure, are the same skills used in production SOC environments every day.

Take the pipeline you built, adapt it to your own log sources, and keep expanding your ATT&CK coverage one detection at a time.

